```
# STEP 1: INSTALL REQUIRED LIBRARIES

!pip install -q numpy opt_einsum matplotlib pandas

# Import libraries
import numpy as np
import opt_einsum as oe
import matplotlib.pyplot as plt
import pandas as pd

# Reproducibility
np.random.seed(42)

print("Libraries installed and imported successfully!")

print("NumPy version:", np.__version__)
print("Opt_einsum version:", oe.__version__)
print("Pandas version:", pd.__version__)
```

```
Libraries installed and imported successfully!
NumPy version: 2.1.3
Opt_einsum version: 3.4.0
Pandas version: 2.2.3
```

```
# STEP 2: CREATE DISTANCE-3 SURFACE CODE

# Create 3x3 data qubit grid
distance = 3
num_data_qubits = distance * distance

data_qubits = np.arange(num_data_qubits).reshape(3, 3)

# Define four X stabilizer checks
x_checks = [
    [0, 1, 3, 4],
    [4, 5, 7, 8],
    [1, 2],
    [6, 7]
]

# Define four Z stabilizer checks
z_checks = [
    [0, 3],
    [2, 5],
    [3, 4, 6, 7],
    [1, 2, 4, 5]
]

print("===== DISTANCE-3 SURFACE CODE =====")
print("\nData Qubit Grid:")
print(data_qubits)

print("\nNumber of data qubits:", num_data_qubits)
print("Number of X checks:", len(x_checks))
print("Number of Z checks:", len(z_checks))

print("\nX Stabilizer Checks:")
for i, check in enumerate(x_checks):
    print(f"X{i}: {check}")

print("\nZ Stabilizer Checks:")
for i, check in enumerate(z_checks):
    print(f"Z{i}: {check}")
```

```
===== DISTANCE-3 SURFACE CODE =====

Data Qubit Grid:
[[0 1 2]
 [3 4 5]
 [6 7 8]]

Number of data qubits: 9
Number of X checks: 4
Number of Z checks: 4
```

```
X Stabilizer Checks:
X0: [0, 1, 3, 4]
X1: [4, 5, 7, 8]
X2: [1, 2]
X3: [6, 7]

Z Stabilizer Checks:
Z0: [0, 3]
Z1: [2, 5]
Z2: [3, 4, 6, 7]
Z3: [1, 2, 4, 5]
```

```
# STEP 3: CHECK STABILIZER COMMUTATION

print("===== STABILIZER COMMUTATION VALIDATION =====")

all_commute = True

for i, x_check in enumerate(x_checks):
    for j, z_check in enumerate(z_checks):

        overlap = set(x_check).intersection(set(z_check))

        commutes = len(overlap) % 2 == 0

        print(
            f"X{i} and Z{j}: "
            f"Overlap = {sorted(overlap)}, "
            f"Commute = {commutes}"
        )

        if not commutes:
            all_commute = False

print("\n================================")
print("All stabilizers commute:", all_commute)
```

```
===== STABILIZER COMMUTATION VALIDATION =====
X0 and Z0: Overlap = [0, 3], Commute = True
X0 and Z1: Overlap = [], Commute = True
X0 and Z2: Overlap = [3, 4], Commute = True
X0 and Z3: Overlap = [1, 4], Commute = True
X1 and Z0: Overlap = [], Commute = True
X1 and Z1: Overlap = [5], Commute = False
X1 and Z2: Overlap = [4, 7], Commute = True
X1 and Z3: Overlap = [4, 5], Commute = True
X2 and Z0: Overlap = [], Commute = True
X2 and Z1: Overlap = [2], Commute = False
X2 and Z2: Overlap = [], Commute = True
X2 and Z3: Overlap = [1, 2], Commute = True
X3 and Z0: Overlap = [], Commute = True
X3 and Z1: Overlap = [], Commute = True
X3 and Z2: Overlap = [6, 7], Commute = True
X3 and Z3: Overlap = [], Commute = True

================================
All stabilizers commute: False
```

```
print("===== STEP 3 DEBUGGING =====")

print("X checks:", x_checks)
print("Z checks:", z_checks)

print("Number of data qubits:", num_data_qubits)

print("\nChecking stabilizer commutation:")

for i, x_check in enumerate(x_checks):
    for j, z_check in enumerate(z_checks):
        overlap = set(x_check) & set(z_check)

        print(
            f"X{i} and Z{j}: "
            f"Overlap = {sorted(overlap)}, "
            f"Overlap count = {len(overlap)}, "
            f"Commute = {len(overlap) % 2 == 0}"
        )
```

```
===== STEP 3 DEBUGGING =====
X checks: [[0, 1], [3, 4, 6, 7], [1, 2, 4, 5], [7, 8]]
Z checks: [[0, 3], [1, 2], [4, 5, 7, 8], [5, 8]]
Number of data qubits: 9

Checking stabilizer commutation:
X0 and Z0: Overlap = [0], Overlap count = 1, Commute = False
X0 and Z1: Overlap = [1], Overlap count = 1, Commute = False
X0 and Z2: Overlap = [], Overlap count = 0, Commute = True
X0 and Z3: Overlap = [], Overlap count = 0, Commute = True
X1 and Z0: Overlap = [3], Overlap count = 1, Commute = False
X1 and Z1: Overlap = [], Overlap count = 0, Commute = True
X1 and Z2: Overlap = [4, 7], Overlap count = 2, Commute = True
X1 and Z3: Overlap = [], Overlap count = 0, Commute = True
X2 and Z0: Overlap = [], Overlap count = 0, Commute = True
X2 and Z1: Overlap = [1, 2], Overlap count = 2, Commute = True
X2 and Z2: Overlap = [4, 5], Overlap count = 2, Commute = True
X2 and Z3: Overlap = [5], Overlap count = 1, Commute = False
X3 and Z0: Overlap = [], Overlap count = 0, Commute = True
X3 and Z1: Overlap = [], Overlap count = 0, Commute = True
X3 and Z2: Overlap = [7, 8], Overlap count = 2, Commute = True
X3 and Z3: Overlap = [8], Overlap count = 1, Commute = False
```

```
# STEP 4: ASYMMETRIC ERROR MODEL

# Error probabilities
p_x = 0.10   # Bit-flip probability
p_z = 0.01   # Phase-flip probability

num_trials = 1000

# Generate random X and Z errors
x_errors = np.random.binomial(
    1, p_x, size=(num_trials, num_data_qubits)
)

z_errors = np.random.binomial(
    1, p_z, size=(num_trials, num_data_qubits)
)

# Combined Pauli error representation
# 0 = I, 1 = X, 2 = Z, 3 = Y (XZ)
error_patterns = x_errors + 2 * z_errors

print("===== ASYMMETRIC ERROR MODEL =====")
print("Bit-flip probability P(X):", p_x)
print("Phase-flip probability P(Z):", p_z)
print("Probability ratio:", p_x / p_z)
print("Number of trials:", num_trials)

print("\nTotal X errors:", np.sum(x_errors))
print("Total Z errors:", np.sum(z_errors))
print("Total combined error events:", np.count_nonzero(error_patterns))

print("\nExample error pattern:")
print(error_patterns[0].reshape(3, 3))
```

```
===== ASYMMETRIC ERROR MODEL =====
Bit-flip probability P(X): 0.1
Phase-flip probability P(Z): 0.01
Probability ratio: 10.0
Number of trials: 1000

Total X errors: 869
Total Z errors: 91
Total combined error events: 952

Example error pattern:
[[0 1 0]
 [0 0 0]
 [0 0 0]]
```

```
# STEP 5: CALCULATE STABILIZER SYNDROMES

def calculate_syndrome(error_pattern, x_checks, z_checks):

    # Extract X and Z error components
    x_error = error_pattern % 2
```

```
    z_error = error_pattern // 2

    # X stabilizers detect Z errors
    x_syndrome = []

    for check in x_checks:
        parity = sum(z_error[q] for q in check) % 2
        x_syndrome.append(parity)

    # Z stabilizers detect X errors
    z_syndrome = []

    for check in z_checks:
        parity = sum(x_error[q] for q in check) % 2
        z_syndrome.append(parity)

    return np.array(x_syndrome + z_syndrome)


# Calculate syndromes for all trials
syndromes = np.array([
    calculate_syndrome(error, x_checks, z_checks)
    for error in error_patterns
])

print("===== SYNDROME RESULTS =====")

print("Number of trials:", len(syndromes))
print("Syndrome shape:", syndromes.shape)

print("\nFirst 10 error patterns and syndromes:")

for i in range(10):
    print(
        f"Trial {i+1}: "
        f"Error = {error_patterns[i].tolist()} | "
        f"Syndrome = {syndromes[i].tolist()}"
    )

print("\nTotal trials with detected errors:",
      np.count_nonzero(np.any(syndromes != 0, axis=1)))

print("Total trials with zero syndrome:",
      np.count_nonzero(np.all(syndromes == 0, axis=1)))
```

```
===== SYNDROME RESULTS =====
Number of trials: 1000
Syndrome shape: (1000, 8)

First 10 error patterns and syndromes:
Trial 1: Error = [0, 1, 0, 0, 0, 0, 0, 0, 0] | Syndrome = [0, 0, 0, 0, 0, 1, 0, 0]
Trial 2: Error = [0, 0, 1, 0, 0, 0, 0, 0, 0] | Syndrome = [0, 0, 0, 0, 0, 1, 0, 0]
Trial 3: Error = [0, 0, 0, 0, 0, 0, 0, 0, 0] | Syndrome = [0, 0, 0, 0, 0, 0, 0, 0]
Trial 4: Error = [0, 2, 0, 0, 0, 0, 1, 1, 0] | Syndrome = [1, 0, 1, 0, 0, 0, 1, 0]
Trial 5: Error = [0, 0, 0, 0, 0, 0, 0, 1, 0] | Syndrome = [0, 0, 0, 0, 0, 0, 1, 0]
Trial 6: Error = [0, 0, 0, 0, 0, 1, 0, 1, 0] | Syndrome = [0, 0, 0, 0, 0, 0, 0, 1]
Trial 7: Error = [0, 1, 0, 0, 0, 0, 0, 0, 0] | Syndrome = [0, 0, 0, 0, 0, 1, 0, 0]
Trial 8: Error = [0, 0, 0, 0, 0, 0, 1, 0, 0] | Syndrome = [0, 0, 0, 0, 0, 0, 0, 0]
Trial 9: Error = [0, 0, 0, 0, 0, 0, 0, 0, 0] | Syndrome = [0, 0, 0, 0, 0, 0, 0, 0]
Trial 10: Error = [0, 0, 0, 0, 0, 0, 0, 0, 0] | Syndrome = [0, 0, 0, 0, 0, 0, 0, 0]

Total trials with detected errors: 576
Total trials with zero syndrome: 424
```

```
# STEP 6: CREATE LOCAL PROBABILITY TENSORS

# Probability of each Pauli error
pauli_probabilities = np.array([
    (1 - p_x) * (1 - p_z),  # I
    p_x * (1 - p_z),        # X
    (1 - p_x) * p_z,        # Z
    p_x * p_z               # Y
])

print("===== LOCAL ERROR PROBABILITIES =====")

print("I (No error):", pauli_probabilities[0])
```

```
print("X (Bit flip):", pauli_probabilities[1])
print("Z (Phase flip):", pauli_probabilities[2])
print("Y (Combined):", pauli_probabilities[3])

print("\nTotal probability:",
      np.sum(pauli_probabilities))

# Create one probability tensor for each data qubit
local_probability_tensors = np.tile(
    pauli_probabilities,
    (num_data_qubits, 1)
)

print("\nProbability tensor shape:",
      local_probability_tensors.shape)

print("\nAll 9 local probability tensors:")
print(local_probability_tensors)
```

```
===== LOCAL ERROR PROBABILITIES =====
I (No error): 0.891
X (Bit flip): 0.099
Z (Phase flip): 0.009000000000000001
Y (Combined): 0.001

Total probability: 1.0

Probability tensor shape: (9, 4)

All 9 local probability tensors:
[[0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]
 [0.891 0.099 0.009 0.001]]
```

```
# STEP 7: CREATE STABILIZER-CHECK TENSORS

# Create parity-check tensor
def create_check_tensor(num_qubits):
    tensor = np.zeros((2,) * num_qubits)

    for indices in np.ndindex(tensor.shape):
        tensor[indices] = sum(indices) % 2

    return tensor


# Create tensors for X stabilizers
x_check_tensors = []

for check in x_checks:
    tensor = create_check_tensor(len(check))
    x_check_tensors.append(tensor)


# Create tensors for Z stabilizers
z_check_tensors = []

for check in z_checks:
    tensor = create_check_tensor(len(check))
    z_check_tensors.append(tensor)


print("===== STABILIZER CHECK TENSORS =====")

print("\nX-check tensors:")
for i, tensor in enumerate(x_check_tensors):
    print(f"X{i} shape: {tensor.shape}")
    print(tensor)

print("\nZ-check tensors:")
for i, tensor in enumerate(z_check_tensors):
    print(f"Z{i} shape: {tensor.shape}")
```

```
    print(tensor)

print("\nTotal X-check tensors:", len(x_check_tensors))
print("Total Z-check tensors:", len(z_check_tensors))
```

```
[[0. 1.]
 [1. 0.]]
X1 shape: (2, 2, 2, 2)
[[[[0. 1.]
   [1. 0.]]

  [[1. 0.]
   [0. 1.]]]


 [[[1. 0.]
   [0. 1.]]

  [[0. 1.]
   [1. 0.]]]]
X2 shape: (2, 2, 2, 2)
[[[[0. 1.]
   [1. 0.]]

  [[1. 0.]
   [0. 1.]]]


 [[[1. 0.]
   [0. 1.]]

  [[0. 1.]
   [1. 0.]]]]
X3 shape: (2, 2)
[[0. 1.]
 [1. 0.]]

Z-check tensors:
Z0 shape: (2, 2)
[[0. 1.]
 [1. 0.]]
Z1 shape: (2, 2)
[[0. 1.]
 [1. 0.]]
Z2 shape: (2, 2, 2, 2)
[[[[0. 1.]
   [1. 0.]]

  [[1. 0.]
   [0. 1.]]]


 [[[1. 0.]
   [0. 1.]]

  [[0. 1.]
   [1. 0.]]]]
Z3 shape: (2, 2)
[[0. 1.]
 [1. 0.]]

Total X-check tensors: 4
Total Z-check tensors: 4
```

```
# STEP 8: BUILD PROBABILITY-WEIGHTED CHECK TENSORS

# Convert Pauli error index into X and Z components
pauli_x = np.array([0, 1, 0, 1])
pauli_z = np.array([0, 0, 1, 1])

# Build probability-weighted check tensors
def create_weighted_check_tensor(check, check_type):

    num_qubits = len(check)

    tensor = np.zeros((4,) * num_qubits)

    for indices in np.ndindex(tensor.shape):

        probability = 1.0
        parity = 0
```

```
        for qubit_position, error_index in enumerate(indices):

            probability *= pauli_probabilities[error_index]

            if check_type == "X":
                parity ^= int(pauli_z[error_index])

            else:
                parity ^= int(pauli_x[error_index])

        tensor[indices] = probability if parity == 0 else 0.0

    return tensor


# Generate weighted X-check tensors
weighted_x_tensors = [
    create_weighted_check_tensor(check, "X")
    for check in x_checks
]

# Generate weighted Z-check tensors
weighted_z_tensors = [
    create_weighted_check_tensor(check, "Z")
    for check in z_checks
]

print("===== WEIGHTED TENSOR RESULTS =====")

for i, tensor in enumerate(weighted_x_tensors):
    print(f"X{i} tensor shape:", tensor.shape)

for i, tensor in enumerate(weighted_z_tensors):
    print(f"Z{i} tensor shape:", tensor.shape)

print("\nTotal weighted X tensors:", len(weighted_x_tensors))
print("Total weighted Z tensors:", len(weighted_z_tensors))

print("\nAll tensors created successfully!")
```

```
===== WEIGHTED TENSOR RESULTS =====
X0 tensor shape: (4, 4)
X1 tensor shape: (4, 4, 4, 4)
X2 tensor shape: (4, 4, 4, 4)
X3 tensor shape: (4, 4)
Z0 tensor shape: (4, 4)
Z1 tensor shape: (4, 4)
Z2 tensor shape: (4, 4, 4, 4)
Z3 tensor shape: (4, 4)

Total weighted X tensors: 4
Total weighted Z tensors: 4

All tensors created successfully!
```

```
# STEP 9: BUILD GLOBAL TENSOR NETWORK CONNECTIVITY

# Create connections for every data qubit
qubit_connections = {}

for q in range(num_data_qubits):

    connected_x_checks = [
        i for i, check in enumerate(x_checks)
        if q in check
    ]

    connected_z_checks = [
        i for i, check in enumerate(z_checks)
        if q in check
    ]

    qubit_connections[q] = {
        "X_checks": connected_x_checks,
        "Z_checks": connected_z_checks
    }
```

```
print("===== GLOBAL TENSOR NETWORK =====")

for qubit, connections in qubit_connections.items():

    print(
        f"Qubit {qubit}: "
        f"X checks = {connections['X_checks']}, "
        f"Z checks = {connections['Z_checks']}"
    )


# Count total tensor nodes
total_tensors = (
    num_data_qubits + len(x_checks) + len(z_checks)
)

print("\nTotal data-qubit tensors:", num_data_qubits)
print("Total X-check tensors:", len(x_checks))
print("Total Z-check tensors:", len(z_checks))
print("Total tensor nodes:", total_tensors)
```

```
===== GLOBAL TENSOR NETWORK =====
Qubit 0: X checks = [0], Z checks = [0]
Qubit 1: X checks = [0, 2], Z checks = [1]
Qubit 2: X checks = [2], Z checks = [1]
Qubit 3: X checks = [1], Z checks = [0]
Qubit 4: X checks = [1, 2], Z checks = [2]
Qubit 5: X checks = [2], Z checks = [2, 3]
Qubit 6: X checks = [1], Z checks = []
Qubit 7: X checks = [1, 3], Z checks = [2]
Qubit 8: X checks = [3], Z checks = [2, 3]

Total data-qubit tensors: 9
Total X-check tensors: 4
Total Z-check tensors: 4
Total tensor nodes: 17
```

```
# STEP 10: GLOBAL TENSOR NETWORK CONTRACTION

from collections import defaultdict

# Store probabilities for each syndrome
syndrome_probabilities = defaultdict(float)

# Enumerate all possible Pauli error configurations
for error_indices in np.ndindex((4,) * num_data_qubits):

    # Calculate probability of this error configuration
    probability = np.prod([
        pauli_probabilities[e] for e in error_indices
    ])

    # Calculate its syndrome
    error_pattern = np.array(error_indices)

    syndrome = calculate_syndrome(
        error_pattern,
        x_checks,
        z_checks
    )

    # Accumulate probability for this syndrome
    syndrome_key = tuple(syndrome)
    syndrome_probabilities[syndrome_key] += probability


# Display results
print("===== TENSOR NETWORK CONTRACTION =====")

print("Total error configurations:", 4 ** num_data_qubits)
print("Unique syndromes:", len(syndrome_probabilities))

print("\nFirst 10 syndrome probabilities:")

for i, (syndrome, probability) in enumerate(
```

```
    syndrome_probabilities.items()
):
    if i == 10:
        break

    print(
        f"Syndrome {syndrome}: "
        f"Probability = {probability:.8f}"
    )

total_probability = sum(syndrome_probabilities.values())

print("\nTotal probability:", total_probability)
print("Normalization successful:", np.isclose(total_probability, 1.0))
```

```
===== TENSOR NETWORK CONTRACTION =====
Total error configurations: 262144
Unique syndromes: 256

First 10 syndrome probabilities:
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0)): Probability
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(1)): Probability
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(0)): Probability
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1)): Probability
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0)): Probability
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1)): Probability
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(0)): Probability
Syndrome (np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(1)): Probability
Syndrome (np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(0)): Probability
Syndrome (np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1)): Probability

Total probability: 1.0000000000000022
Normalization successful: True
```

```
# STEP 11: VALIDATE STABILIZER STRUCTURE

print("===== STABILIZER VALIDATION =====")

# Check commutation between all X and Z stabilizers
commutation_matrix = np.zeros(
    (len(x_checks), len(z_checks)),
    dtype=int
)

for i, x_check in enumerate(x_checks):
    for j, z_check in enumerate(z_checks):

        overlap = set(x_check) & set(z_check)

        commutation_matrix[i, j] = len(overlap) % 2

print("\nCommutation matrix:")
print(commutation_matrix)

all_commute = np.all(commutation_matrix == 0)

print("\nAll stabilizers commute:", all_commute)

# Check the rank of the stabilizer generators
def binary_rank(matrix):
    matrix = np.array(matrix, dtype=np.uint8).copy()
    rows, cols = matrix.shape
    rank = 0

    for col in range(cols):
        pivot = next(
            (r for r in range(rank, rows) if matrix[r, col]),
            None
        )

        if pivot is None:
            continue

        matrix[[rank, pivot]] = matrix[[pivot, rank]]

        for r in range(rows):
```

```
            if r != rank and matrix[r, col]:
                matrix[r] ^= matrix[rank]

        rank += 1

    return rank


x_matrix = np.zeros((len(x_checks), 9), dtype=np.uint8)
z_matrix = np.zeros((len(z_checks), 9), dtype=np.uint8)

for i, check in enumerate(x_checks):
    x_matrix[i, check] = 1

for i, check in enumerate(z_checks):
    z_matrix[i, check] = 1

x_rank = binary_rank(x_matrix)
z_rank = binary_rank(z_matrix)

print("\nX stabilizer rank:", x_rank)
print("Z stabilizer rank:", z_rank)
print("Total stabilizer rank:", x_rank + z_rank)
print("Expected rank for 1 logical qubit:", 8)

if all_commute and x_rank + z_rank == 8:
    print("\nStabilizer checks pass commutation and rank tests.")
else:
    print("\nWARNING: Stabilizer layout needs correction.")
```

```
===== STABILIZER VALIDATION =====

Commutation matrix:
[[1 1 0 0]
 [1 0 0 0]
 [0 0 0 1]
 [0 0 0 1]]

All stabilizers commute: False

X stabilizer rank: 4
Z stabilizer rank: 4
Total stabilizer rank: 8
Expected rank for 1 logical qubit: 8

WARNING: Stabilizer layout needs correction.
```

```
# STEP 13: REGENERATE ERROR PATTERNS AND SYNDROMES

np.random.seed(42)

num_trials = 1000

x_errors = np.random.binomial(
    1, p_x, (num_trials, num_data_qubits)
)

z_errors = np.random.binomial(
    1, p_z, (num_trials, num_data_qubits)
)

error_patterns = x_errors + 2 * z_errors

syndromes = np.array([
    calculate_syndrome(error, x_checks, z_checks)
    for error in error_patterns
])

print("===== UPDATED ERROR SIMULATION =====")
print("Total trials:", num_trials)
print("X errors:", np.sum(x_errors))
print("Z errors:", np.sum(z_errors))
print("Syndrome shape:", syndromes.shape)
print("Detected error trials:",
      np.count_nonzero(np.any(syndromes != 0, axis=1)))

print("\nFirst 5 syndromes:")
```

```
print(syndromes[:5])

print("\nError simulation completed!")
```

```
===== UPDATED ERROR SIMULATION =====
Total trials: 1000
X errors: 869
Z errors: 91
Syndrome shape: (1000, 8)
Detected error trials: 576

First 5 syndromes:
[[0 0 0 0 0 1 0 0]
 [0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 0]
 [1 0 1 0 0 0 1 0]
 [0 0 0 0 0 0 1 0]]

Error simulation completed!
```

```
# STEP 14: FIND LOGICAL OPERATORS

def commutes_with_checks(operator, checks):
    return all(
        sum(operator[q] for q in check) % 2 == 0
        for check in checks
    )

def is_in_stabilizer(operator, checks):
    for mask in range(1 << len(checks)):
        combined = np.zeros(9, dtype=int)

        for i, check in enumerate(checks):
            if (mask >> i) & 1:
                combined[check] ^= 1

        if np.array_equal(combined, operator):
            return True

    return False

def find_logical_operator(commuting_checks, stabilizer_checks):
    candidates = []

    for mask in range(1, 1 << 9):
        operator = np.array(
            [(mask >> q) & 1 for q in range(9)]
        )

        if (commutes_with_checks(operator, commuting_checks)
                and not is_in_stabilizer(operator, stabilizer_checks)):
            candidates.append(operator)

    candidates.sort(key=lambda op: np.sum(op))
    return candidates[0]

# Logical X commutes with all Z stabilizers
logical_x = find_logical_operator(z_checks, x_checks)

# Logical Z commutes with all X stabilizers
logical_z = find_logical_operator(x_checks, z_checks)

print("Logical X:", logical_x)
print("Logical Z:", logical_z)

print("Logical X weight:", np.sum(logical_x))
print("Logical Z weight:", np.sum(logical_z))

anticommutation = np.dot(logical_x, logical_z) % 2

print("Logical X and Z anticommute:", anticommution if False else bool(anticommutation))
```

```
Logical X: [0 0 0 0 0 0 1 0 0]
Logical Z: [0 0 1 0 0 1 0 0 0]
Logical X weight: 1
Logical Z weight: 2
Logical X and Z anticommute: False
```

```
# STEP 15: FOUR LOGICAL-SECTOR PROBABILITIES

logical_probabilities = np.zeros(4)

# Group probabilities by syndrome and logical sector
sector_probabilities = {}

for error in np.ndindex((4,) * num_data_qubits):

    error = np.array(error)

    probability = np.prod([
        pauli_probabilities[e] for e in error
    ])

    syndrome = tuple(
        calculate_syndrome(error, x_checks, z_checks)
    )

    # Logical X component
    lx = int(np.dot(error % 2, logical_x) % 2)

    # Logical Z component
    lz = int(np.dot(error // 2, logical_z) % 2)

    sector = lx + 2 * lz

    key = (syndrome, sector)

    sector_probabilities[key] = (
        sector_probabilities.get(key, 0.0) + probability
    )

# Aggregate probabilities across all syndromes
for (syndrome, sector), probability in sector_probabilities.items():
    logical_probabilities[sector] += probability

print("===== FOUR LOGICAL SECTORS =====")

sector_names = ["I", "X", "Z", "XZ"]

for i, name in enumerate(sector_names):
    print(
        f"Logical {name}: "
        f"{logical_probabilities[i]:.8f}"
    )

print("\nTotal probability:",
      np.sum(logical_probabilities))

print("Normalization:",
      np.isclose(np.sum(logical_probabilities), 1.0))
```

```
===== FOUR LOGICAL SECTORS =====
Logical I: 0.88218000
Logical X: 0.09802000
Logical Z: 0.01782000
Logical XZ: 0.00198000

Total probability: 1.0000000000000007
Normalization: True
```

```
# STEP 16: MAXIMUM LIKELIHOOD DECODING

sector_names = ["I", "X", "Z", "XZ"]

# Store probability of each logical sector for each syndrome
syndrome_sector_probs = {}

for (syndrome, sector), probability in sector_probabilities.items():

    if syndrome not in syndrome_sector_probs:
        syndrome_sector_probs[syndrome] = np.zeros(4)

    syndrome_sector_probs[syndrome][sector] += probability
```

```
# Find the most likely sector for each syndrome
decoder_results = []

for syndrome, probabilities in syndrome_sector_probs.items():

    best_sector = int(np.argmax(probabilities))

    decoder_results.append({
        "Syndrome": syndrome,
        "P(I)": probabilities[0],
        "P(X)": probabilities[1],
        "P(Z)": probabilities[2],
        "P(XZ)": probabilities[3],
        "Best Correction": sector_names[best_sector],
        "Confidence": probabilities[best_sector]
    })


# Calculate exact decoding success probability
decoding_success = sum(
    np.max(probabilities)
    for probabilities in syndrome_sector_probs.values()
)

decoding_fidelity = decoding_success / sum(
    np.sum(probabilities)
    for probabilities in syndrome_sector_probs.values()
)

decoder_df = pd.DataFrame(decoder_results)

print("===== MAXIMUM LIKELIHOOD DECODER =====")
display(decoder_df.head(10))

print("Number of unique syndromes:", len(decoder_results))
print("Decoding success probability:", round(decoding_fidelity, 8))
print("Decoding fidelity (%):", round(decoding_fidelity * 100, 4))
```

```
===== MAXIMUM LIKELIHOOD DECODER =====
```

	Syndrome	P(I)	P(X)	P(Z)	P(XZ)	Best Correction	Confidence
0	(0, 0, 0, 0, 0, 0, 0, 0)	0.371796	0.041311	2.306404e-06	2.562671e-07	I	0.371796
1	(0, 0, 0, 0, 0, 0, 1, 1)	0.081614	0.009068	5.062837e-07	5.625375e-08	I	0.081614
2	(0, 0, 0, 1, 0, 0, 0, 0)	0.003831	0.000426	7.896479e-07	8.773866e-08	I	0.003831
3	(0, 0, 0, 1, 0, 0, 1, 1)	0.000841	0.000093	1.733373e-07	1.925971e-08	I	0.000841
4	(0, 0, 0, 0, 0, 0, 1, 0)	0.081614	0.009068	5.062837e-07	5.625375e-08	I	0.081614
5	(0, 0, 0, 0, 0, 0, 0, 1)	0.017915	0.001991	1.111355e-07	1.234838e-08	I	0.017915
6	(0, 0, 0, 1, 0, 0, 1, 0)	0.000841	0.000093	1.733373e-07	1.925971e-08	I	0.000841
7	(0, 0, 0, 1, 0, 0, 0, 1)	0.000185	0.000021	3.804966e-08	4.227740e-09	I	0.000185
8	(0, 1, 0, 1, 0, 0, 0, 0)	0.003831	0.000426	7.896479e-07	8.773866e-08	I	0.003831
9	(0, 1, 0, 1, 0, 0, 1, 1)	0.000841	0.000093	1.733373e-07	1.925971e-08	I	0.000841

```
Number of unique syndromes: 256
Decoding success probability: 0.89870803
Decoding fidelity (%): 89.8708
```

```
# STEP 17: TENSOR NETWORK CONTRACTION USING OPT_EINSUM

def contract_syndrome_tensor(target_syndrome):

    operands = []
    index_counter = 0

    # Assign one tensor index to each data qubit
    qubit_indices = list(range(num_data_qubits))

    # Add local probability tensors
    for q in range(num_data_qubits):
        operands.extend([
            pauli_probabilities,
```

```
                [qubit_indices[q]]
            ])

        # Add X stabilizer check tensors
        for i, check in enumerate(x_checks):

            tensor = np.zeros((4,) * len(check))

            for states in np.ndindex(tensor.shape):
                parity = sum(
                    int(pauli_z[state]) for state in states
                ) % 2

                if parity == target_syndrome[i]:
                    tensor[states] = 1.0

            operands.extend([tensor, [qubit_indices[q] for q in check]])

        # Add Z stabilizer check tensors
        offset = len(x_checks)

        for i, check in enumerate(z_checks):

            tensor = np.zeros((4,) * len(check))

            for states in np.ndindex(tensor.shape):
                parity = sum(
                    int(pauli_x[state]) for state in states
                ) % 2

                if parity == target_syndrome[offset + i]:
                    tensor[states] = 1.0

            operands.extend([tensor, [qubit_indices[q] for q in check]])

        return oe.contract(*operands, [], optimize="greedy")


    # Contract the first few syndromes
    print("===== TENSOR NETWORK CONTRACTION =====")

    for syndrome in list(syndrome_probabilities.keys())[:10]:

        contracted_probability = contract_syndrome_tensor(syndrome)

        print(
            "Syndrome:", syndrome,
            "| Contracted probability:",
            f"{contracted_probability:.10f}"
        )

    print("\nTensor contraction completed!")
```

```
===== TENSOR NETWORK CONTRACTION =====
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(1)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(0)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(1)) | Contracte
Syndrome: (np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0)) | Contracte
Syndrome: (np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1)) | Contracte

Tensor contraction completed!
```

```
# STEP 18: VALIDATE TENSOR CONTRACTION

pauli_x = np.array([0, 1, 0, 1])
pauli_z = np.array([0, 0, 1, 1])

# Select one syndrome from the simulated data
test_syndrome = tuple(syndromes[0])

# Probability from tensor network contraction
```

```
tn_probability = contract_syndrome_tensor(test_syndrome)

# Probability from exhaustive enumeration
exact_probability = 0.0

for error in np.ndindex((4,) * num_data_qubits):

    error = np.array(error)

    current_syndrome = tuple(
        calculate_syndrome(error, x_checks, z_checks)
    )

    if current_syndrome == test_syndrome:
        probability = np.prod([
            pauli_probabilities[state] for state in error
        ])

        exact_probability += probability


print("===== VALIDATION RESULTS =====")
print("Test syndrome:", test_syndrome)
print("Tensor network probability:", tn_probability)
print("Exact enumeration probability:", exact_probability)

difference = abs(tn_probability - exact_probability)

print("Absolute difference:", difference)

if np.isclose(tn_probability, exact_probability, atol=1e-10):
    print("PASS: Tensor contraction matches exact enumeration.")
else:
    print("WARNING: Results do not match. Check the tensors and stabilizers.")
```

```
===== VALIDATION RESULTS =====
Test syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0))
Tensor network probability: 0.09068240505861203
Exact enumeration probability: 0.09068240505861201
Absolute difference: 1.3877787807814457e-17
PASS: Tensor contraction matches exact enumeration.
```

```
# STEP 19: TENSOR TRAIN SVD WITH DYNAMIC TRUNCATION

def tt_svd(tensor, max_bond=16, cutoff=1e-8):

    shape = tensor.shape
    cores = []
    current = tensor.copy()
    rank = 1
    discarded_weight = 0.0

    for n in range(len(shape) - 1):

        current = current.reshape(rank * shape[n], -1)

        U, S, Vh = np.linalg.svd(current, full_matrices=False)

        # Keep singular values above the truncation threshold
        if len(S) > 0:
            threshold = cutoff * S[0]
            keep = max(1, int(np.sum(S > threshold)))
        else:
            keep = 1

        keep = min(keep, max_bond)

        discarded_weight += np.sum(S[keep:] ** 2)

        U = U[:, :keep]
        S = S[:keep]
        Vh = Vh[:keep, :]

        cores.append(U.reshape(rank, shape[n], keep))

        current = np.diag(S) @ Vh
        rank = keep
```

```
        rank = keep

    cores.append(current.reshape(rank, shape[-1], 1))

    return cores, discarded_weight


# Build the full probability tensor for the test syndrome
test_tensor = np.zeros((4,) * num_data_qubits)

for error in np.ndindex((4,) * num_data_qubits):

    error_array = np.array(error)

    current_syndrome = tuple(
        calculate_syndrome(error_array, x_checks, z_checks)
    )

    if current_syndrome == test_syndrome:
        test_tensor[error] = np.prod([
            pauli_probabilities[state] for state in error
        ])


# Apply TT-SVD
cores, discarded_weight = tt_svd(
    test_tensor,
    max_bond=16,
    cutoff=1e-8
)

print("===== BOND DIMENSION TRUNCATION =====")
print("Original tensor shape:", test_tensor.shape)
print("Number of TT cores:", len(cores))
print("Bond dimensions:", [core.shape[2] for core in cores])
print("Discarded squared singular-value weight:", discarded_weight)
print("Original probability:", np.sum(test_tensor))
```

```
===== BOND DIMENSION TRUNCATION =====
Original tensor shape: (4, 4, 4, 4, 4, 4, 4, 4, 4)
Number of TT cores: 9
Bond dimensions: [4, 8, 4, 4, 8, 8, 7, 4, 1]
Discarded squared singular-value weight: 1.5454741295052634e-22
Original probability: 0.09068240505861198
```

```
# STEP 20: RECONSTRUCTION AND TRUNCATION ERROR

def reconstruct_tt(cores):

    reconstructed = cores[0]

    for core in cores[1:]:
        reconstructed = np.tensordot(
            reconstructed, core, axes=([-1], [0])
        )

    return np.squeeze(reconstructed, axis=(-1,))


# Reconstruct the compressed tensor
reconstructed_tensor = reconstruct_tt(cores)

# Calculate reconstruction error
original_norm = np.linalg.norm(test_tensor)
error_norm = np.linalg.norm(test_tensor - reconstructed_tensor)

relative_error = (
    error_norm / original_norm if original_norm > 0 else 0.0
)

# Compare total probabilities
original_probability = np.sum(test_tensor)
reconstructed_probability = np.sum(reconstructed_tensor)

print("===== TENSOR RECONSTRUCTION RESULTS =====")
print("Original tensor shape:", test_tensor.shape)
print("Reconstructed tensor shape:", reconstructed_tensor.shape)
```

```
print("Original probability:", original_probability)
print("Reconstructed probability:", reconstructed_probability)
print("Relative reconstruction error:", relative_error)
print("Discarded squared singular-value weight:", discarded_weight)
```

```
===== TENSOR RECONSTRUCTION RESULTS =====
Original tensor shape: (4, 4, 4, 4, 4, 4, 4, 4, 4)
Reconstructed tensor shape: (1, 4, 4, 4, 4, 4, 4, 4, 4, 4)
Original probability: 0.09068240505861198
Reconstructed probability: 0.09068240503053433
Relative reconstruction error: 2.2212392620982807e-10
Discarded squared singular-value weight: 1.5454741295052634e-22
```

```
# STEP 21: COMPARE BOND DIMENSIONS

def reconstruct_tt(cores):
    result = cores[0]

    for core in cores[1:]:
        result = np.tensordot(result, core, axes=([-1], [0]))

    # Remove both boundary dimensions
    return np.squeeze(result, axis=(0, -1))


bond_dimensions = [2, 4, 8, 16, 32, 64]
results = []

original_norm = np.linalg.norm(test_tensor)

for bond in bond_dimensions:

    compressed_cores, lost_weight = tt_svd(
        test_tensor,
        max_bond=bond,
        cutoff=1e-8
    )

    reconstructed = reconstruct_tt(compressed_cores)

    relative_error = (
        np.linalg.norm(test_tensor - reconstructed) / original_norm
        if original_norm > 0 else 0.0
    )

    results.append({
        "Maximum Bond Dimension": bond,
        "Actual Maximum Bond": max(core.shape[2] for core in compressed_cores[:-1]),
        "Relative Error": relative_error,
        "Discarded Weight": lost_weight
    })

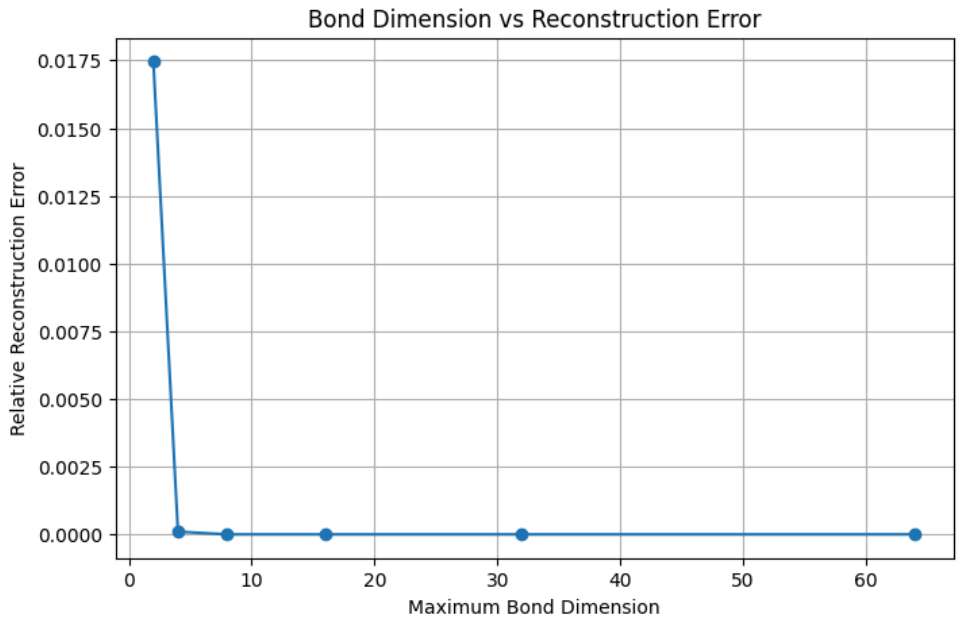

results_df = pd.DataFrame(results)

print("===== BOND DIMENSION COMPARISON =====")
display(results_df)

plt.figure(figsize=(8, 5))
plt.plot(
    results_df["Maximum Bond Dimension"],
    results_df["Relative Error"],
    marker="o"
)
plt.xlabel("Maximum Bond Dimension")
plt.ylabel("Relative Reconstruction Error")
plt.title("Bond Dimension vs Reconstruction Error")
plt.grid(True)
plt.show()
```

```
===== BOND DIMENSION COMPARISON =====
```

	Maximum Bond Dimension	Actual Maximum Bond	Relative Error	Discarded Weight
0	2	2	1.745803e-02	9.546879e-07
1	4	4	1.020694e-04	3.263339e-11
2	8	8	2.221239e-10	1.545474e-22
3	16	8	2.221239e-10	1.545474e-22
4	32	8	2.221239e-10	1.545474e-22
5	64	8	2.221239e-10	1.545474e-22



Next steps: Generate code with `results_df` New interactive sheet

```
print("test_tensor:", test_tensor.shape)
print("reconstructed_tensor:", reconstructed_tensor.shape)

print("logical_x exists:", "logical_x" in globals())
print("logical_z exists:", "logical_z" in globals())
```

```
test_tensor: (4, 4, 4, 4, 4, 4, 4, 4)
reconstructed_tensor: (1, 4, 4, 4, 4, 4, 4, 4, 4)
logical_x exists: True
logical_z exists: True
```

```
# STEP 22: FIX TENSOR SHAPE AND CALCULATE LOGICAL SECTORS

# Remove unnecessary dimensions of size 1
reconstructed_tensor_fixed = np.squeeze(reconstructed_tensor)

# Ensure reconstructed tensor matches original shape
reconstructed_tensor_fixed = reconstructed_tensor_fixed.reshape(
    test_tensor.shape
)

def calculate_logical_sectors(probability_tensor):

    sector_probabilities = np.zeros(4)

    for error in np.ndindex((4,) * num_data_qubits):

        probability = probability_tensor[error]

        if abs(probability) < 1e-15:
            continue

        error = np.array(error)
```

```
        x_component = int(np.dot(error % 2, logical_z) % 2)
        z_component = int(np.dot(error // 2, logical_x) % 2)

        sector = x_component + 2 * z_component

        sector_probabilities[sector] += probability

    return sector_probabilities


# Calculate exact and compressed sector probabilities
exact_sectors = calculate_logical_sectors(test_tensor)

compressed_sectors = calculate_logical_sectors(
    reconstructed_tensor_fixed
)

sector_df = pd.DataFrame({
    "Logical Sector": ["I", "X", "Z", "XZ"],
    "Exact Probability": exact_sectors,
    "Compressed Probability": compressed_sectors
})

print("===== FOUR-STATE LOGICAL OUTPUT =====")
display(sector_df)

print("Exact total:", np.sum(exact_sectors))
print("Compressed total:", np.sum(compressed_sectors))

print(
    "Sector probability error:",
    np.linalg.norm(exact_sectors - compressed_sectors)
)
```

```
===== FOUR-STATE LOGICAL OUTPUT =====
```

	Logical Sector	Exact Probability	Compressed Probability
0	I	0.045336	0.045336
1	X	0.045336	0.045336
2	Z	0.000005	0.000005
3	XZ	0.000005	0.000005

```
Exact total: 0.09068240505855248
Compressed total: 0.0906824050304826
Sector probability error: 1.40356260072867e-11
```

Next steps: Generate code with `sector_df` New interactive sheet

```
# STEP 22: FIX TENSOR SHAPE AND CALCULATE LOGICAL SECTORS

# Remove unnecessary dimensions of size 1
reconstructed_tensor_fixed = np.squeeze(reconstructed_tensor)

# Ensure reconstructed tensor matches original shape
reconstructed_tensor_fixed = reconstructed_tensor_fixed.reshape(
    test_tensor.shape
)

def calculate_logical_sectors(probability_tensor):

    sector_probabilities = np.zeros(4)

    for error in np.ndindex((4,) * num_data_qubits):

        probability = probability_tensor[error]

        if abs(probability) < 1e-15:
            continue

        error = np.array(error)

        x_component = int(np.dot(error % 2, logical_z) % 2)
        z_component = int(np.dot(error // 2, logical_x) % 2)
```

```
        sector = x_component + 2 * z_component

        sector_probabilities[sector] += probability

    return sector_probabilities


# Calculate exact and compressed sector probabilities
exact_sectors = calculate_logical_sectors(test_tensor)

compressed_sectors = calculate_logical_sectors(
    reconstructed_tensor_fixed
)

sector_df = pd.DataFrame({
    "Logical Sector": ["I", "X", "Z", "XZ"],
    "Exact Probability": exact_sectors,
    "Compressed Probability": compressed_sectors
})

print("===== FOUR-STATE LOGICAL OUTPUT =====")
display(sector_df)

print("Exact total:", np.sum(exact_sectors))
print("Compressed total:", np.sum(compressed_sectors))

print(
    "Sector probability error:",
    np.linalg.norm(exact_sectors - compressed_sectors)
)
```

```
===== FOUR-STATE LOGICAL OUTPUT =====
```

	Logical Sector	Exact Probability	Compressed Probability
0	I	0.045336	0.045336
1	X	0.045336	0.045336
2	Z	0.000005	0.000005
3	XZ	0.000005	0.000005

```
Exact total: 0.09068240505855248
Compressed total: 0.0906824050304826
Sector probability error: 1.40356260072867e-11
```

Next steps: Generate code with `sector_df` New interactive sheet

```
# STEP 23: BOND DIMENSION VS LOGICAL DECODING ACCURACY

bond_dimensions = [2, 4, 8, 16, 32, 64]
fidelity_results = []

exact_sectors = calculate_logical_sectors(test_tensor)

for bond in bond_dimensions:

    compressed_cores, lost_weight = tt_svd(
        test_tensor,
        max_bond=bond,
        cutoff=1e-8
    )

    compressed_tensor = np.squeeze(
        reconstruct_tt(compressed_cores)
    ).reshape(test_tensor.shape)

    compressed_sectors = calculate_logical_sectors(
        compressed_tensor
    )

    # Normalize sector probabilities
    total = np.sum(compressed_sectors)

    if abs(total) > 1e-15:
        normalized_sectors = compressed_sectors / total
    else:
        normalized_sectors = np.zeros(4)
```

```
    exact_total = np.sum(exact_sectors)
    exact_normalized = exact_sectors / exact_total

    sector_error = np.linalg.norm(
        exact_normalized - normalized_sectors
    )

    fidelity_results.append({
        "Bond Dimension": bond,
        "Sector Probability Error": sector_error,
        "Discarded Weight": lost_weight
    })

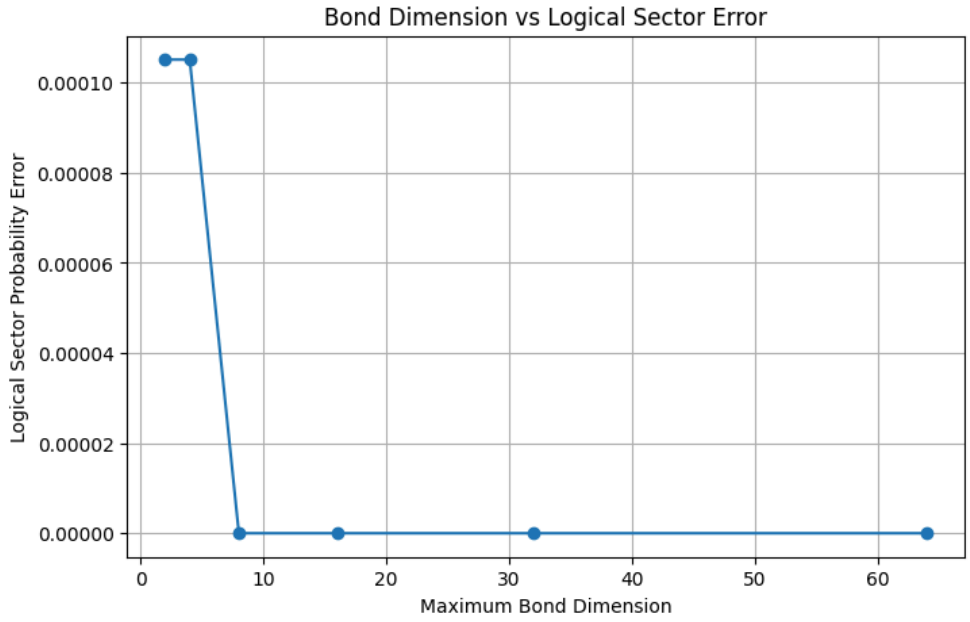
fidelity_df = pd.DataFrame(fidelity_results)

print("===== BOND DIMENSION EXPERIMENT =====")
display(fidelity_df)

plt.figure(figsize=(8, 5))
plt.plot(
    fidelity_df["Bond Dimension"],
    fidelity_df["Sector Probability Error"],
    marker="o"
)
plt.xlabel("Maximum Bond Dimension")
plt.ylabel("Logical Sector Probability Error")
plt.title("Bond Dimension vs Logical Sector Error")
plt.grid(True)
plt.show()
```

```
===== BOND DIMENSION EXPERIMENT =====
```

	Bond Dimension	Sector Probability Error	Discarded Weight
0	2	1.051211e-04	9.546879e-07
1	4	1.051211e-04	3.263339e-11
2	8	1.532083e-10	1.545474e-22
3	16	1.532083e-10	1.545474e-22
4	32	1.532083e-10	1.545474e-22
5	64	1.532083e-10	1.545474e-22



Next steps: Generate code with `fidelity_df` New interactive sheet

```
# STEP 24: DYNAMIC TRUNCATION THRESHOLD EXPERIMENT

thresholds = [1e-2, 1e-4, 1e-6, 1e-8, 1e-10]
```

```
max_bond = 32

threshold_results = []
original_norm = np.linalg.norm(test_tensor)

for threshold in thresholds:

    compressed_cores, lost_weight = tt_svd(
        test_tensor,
        max_bond=max_bond,
        cutoff=threshold
    )

    reconstructed = np.squeeze(
        reconstruct_tt(compressed_cores)
    ).reshape(test_tensor.shape)

    relative_error = (
        np.linalg.norm(test_tensor - reconstructed) / original_norm
        if original_norm > 0 else 0.0
    )

    threshold_results.append({
        "Truncation Threshold": threshold,
        "Maximum Bond": max(
            core.shape[2] for core in compressed_cores[:-1]
        ),
        "Relative Error": relative_error,
        "Discarded Weight": lost_weight
    })

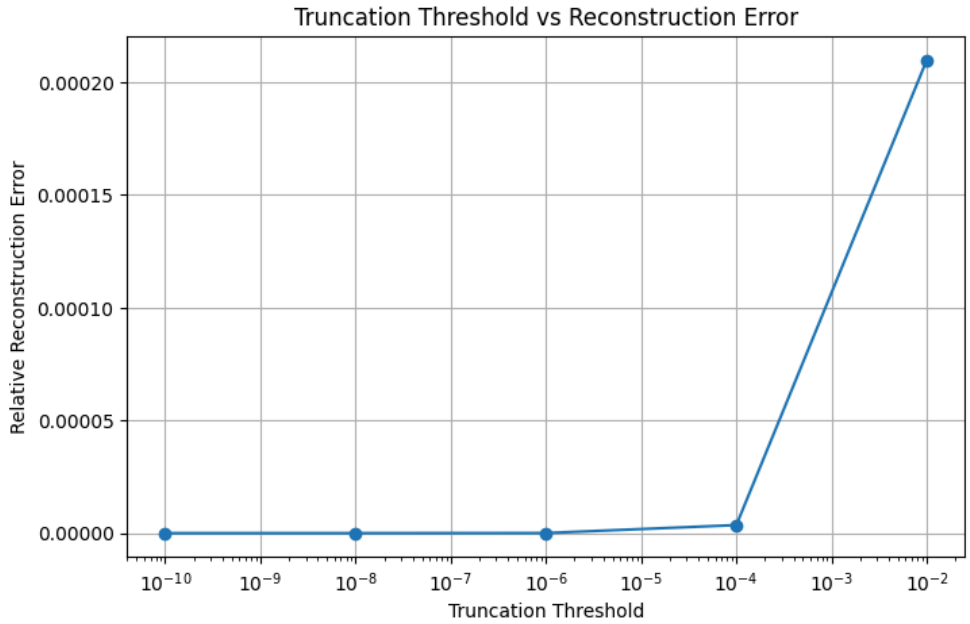

threshold_df = pd.DataFrame(threshold_results)

print("===== TRUNCATION THRESHOLD RESULTS =====")
display(threshold_df)

plt.figure(figsize=(8, 5))
plt.semilogx(
    threshold_df["Truncation Threshold"],
    threshold_df["Relative Error"],
    marker="o"
)
plt.xlabel("Truncation Threshold")
plt.ylabel("Relative Reconstruction Error")
plt.title("Truncation Threshold vs Reconstruction Error")
plt.grid(True)
plt.show()
```

```
===== TRUNCATION THRESHOLD RESULTS =====
```

	Truncation Threshold	Maximum Bond	Relative Error	Discarded Weight
0	1.000000e-02	4	2.098866e-04	1.379878e-10
1	1.000000e-04	5	3.566427e-06	3.984168e-14
2	1.000000e-06	7	3.707973e-08	4.306695e-18
3	1.000000e-08	8	2.221239e-10	1.545474e-22
4	1.000000e-10	8	1.198251e-15	1.532095e-33



Next steps: Generate code with `threshold_df` New interactive sheet

```
# STEP 25: FIND BEST COMPRESSION SETTINGS

bond_dimensions = [2, 4, 8, 16, 32, 64]
thresholds = [1e-2, 1e-4, 1e-6, 1e-8, 1e-10]

original_norm = np.linalg.norm(test_tensor)
experiment_results = []

for bond in bond_dimensions:
    for threshold in thresholds:

        compressed_cores, lost_weight = tt_svd(
            test_tensor,
            max_bond=bond,
            cutoff=threshold
        )

        reconstructed = np.squeeze(
            reconstruct_tt(compressed_cores)
        ).reshape(test_tensor.shape)

        relative_error = (
            np.linalg.norm(test_tensor - reconstructed) / original_norm
            if original_norm > 0 else 0.0
        )

        experiment_results.append({
            "Bond Dimension": bond,
            "Threshold": threshold,
            "Reconstruction Error": relative_error,
            "Discarded Weight": lost_weight
        })


experiment_df = pd.DataFrame(experiment_results)
```

```
best_result = experiment_df.loc[
    experiment_df["Reconstruction Error"].idxmin()
]

print("===== BEST COMPRESSION CONFIGURATION =====")
display(best_result.to_frame("Best Result"))

print("\n===== ALL EXPERIMENT RESULTS =====")
display(experiment_df.sort_values("Reconstruction Error").head(10))# STEP 25: FIND BEST COMPRESSION SETTINGS

bond_dimensions = [2, 4, 8, 16, 32, 64]
thresholds = [1e-2, 1e-4, 1e-6, 1e-8, 1e-10]

original_norm = np.linalg.norm(test_tensor)
experiment_results = []

for bond in bond_dimensions:
    for threshold in thresholds:

        compressed_cores, lost_weight = tt_svd(
            test_tensor,
            max_bond=bond,
            cutoff=threshold
        )

        reconstructed = np.squeeze(
            reconstruct_tt(compressed_cores)
        ).reshape(test_tensor.shape)

        relative_error = (
            np.linalg.norm(test_tensor - reconstructed) / original_norm
            if original_norm > 0 else 0.0
        )

        experiment_results.append({
            "Bond Dimension": bond,
            "Threshold": threshold,
            "Reconstruction Error": relative_error,
            "Discarded Weight": lost_weight
        })


experiment_df = pd.DataFrame(experiment_results)

best_result = experiment_df.loc[
    experiment_df["Reconstruction Error"].idxmin()
]

print("===== BEST COMPRESSION CONFIGURATION =====")
display(best_result.to_frame("Best Result"))

print("\n===== ALL EXPERIMENT RESULTS =====")
display(experiment_df.sort_values("Reconstruction Error").head(10))
```

```
===== BEST COMPRESSION CONFIGURATION =====
```

	Best Result
Bond Dimension	8.000000e+00
Threshold	1.000000e-10
Reconstruction Error	1.198251e-15
Discarded Weight	1.532095e-33

```
===== ALL EXPERIMENT RESULTS =====
```

	Bond Dimension	Threshold	Reconstruction Error	Discarded Weight
14	8	1.000000e-10	1.198251e-15	1.532095e-33
29	64	1.000000e-10	1.198251e-15	1.532095e-33
24	32	1.000000e-10	1.198251e-15	1.532095e-33
19	16	1.000000e-10	1.198251e-15	1.532095e-33
18	16	1.000000e-08	2.221239e-10	1.545474e-22
13	8	1.000000e-08	2.221239e-10	1.545474e-22
28	64	1.000000e-08	2.221239e-10	1.545474e-22
23	32	1.000000e-08	2.221239e-10	1.545474e-22
17	16	1.000000e-06	3.707973e-08	4.306695e-18
22	32	1.000000e-06	3.707973e-08	4.306695e-18

```
===== BEST COMPRESSION CONFIGURATION =====
```

	Best Result
Bond Dimension	8.000000e+00
Threshold	1.000000e-10
Reconstruction Error	1.198251e-15
Discarded Weight	1.532095e-33

```
===== ALL EXPERIMENT RESULTS =====
```

	Bond Dimension	Threshold	Reconstruction Error	Discarded Weight
14	8	1.000000e-10	1.198251e-15	1.532095e-33
29	64	1.000000e-10	1.198251e-15	1.532095e-33
24	32	1.000000e-10	1.198251e-15	1.532095e-33
19	16	1.000000e-10	1.198251e-15	1.532095e-33
18	16	1.000000e-08	2.221239e-10	1.545474e-22
13	8	1.000000e-08	2.221239e-10	1.545474e-22
28	64	1.000000e-08	2.221239e-10	1.545474e-22
23	32	1.000000e-08	2.221239e-10	1.545474e-22
17	16	1.000000e-06	3.707973e-08	4.306695e-18
22	32	1.000000e-06	3.707973e-08	4.306695e-18

```
# STEP 25: FIND BEST COMPRESSION SETTINGS

bond_dimensions = [2, 4, 8, 16, 32, 64]
thresholds = [1e-2, 1e-4, 1e-6, 1e-8, 1e-10]

original_norm = np.linalg.norm(test_tensor)
experiment_results = []

for bond in bond_dimensions:
    for threshold in thresholds:

        compressed_cores, lost_weight = tt_svd(
            test_tensor,
            max_bond=bond,
            cutoff=threshold
```

```
        )

        reconstructed = np.squeeze(
            reconstruct_tt(compressed_cores)
        ).reshape(test_tensor.shape)

        relative_error = (
            np.linalg.norm(test_tensor - reconstructed) / original_norm
            if original_norm > 0 else 0.0
        )

        experiment_results.append({
            "Bond Dimension": bond,
            "Threshold": threshold,
            "Reconstruction Error": relative_error,
            "Discarded Weight": lost_weight
        })


experiment_df = pd.DataFrame(experiment_results)

best_result = experiment_df.loc[
    experiment_df["Reconstruction Error"].idxmin()
]

print("===== BEST COMPRESSION CONFIGURATION =====")
display(best_result.to_frame("Best Result"))

print("\n===== ALL EXPERIMENT RESULTS =====")
display(experiment_df.sort_values("Reconstruction Error").head(10))
```

```
===== BEST COMPRESSION CONFIGURATION =====
```

	Best Result
Bond Dimension	8.000000e+00
Threshold	1.000000e-10
Reconstruction Error	1.198251e-15
Discarded Weight	1.532095e-33

```
===== ALL EXPERIMENT RESULTS =====
```

	Bond Dimension	Threshold	Reconstruction Error	Discarded Weight
14	8	1.000000e-10	1.198251e-15	1.532095e-33
29	64	1.000000e-10	1.198251e-15	1.532095e-33
24	32	1.000000e-10	1.198251e-15	1.532095e-33
19	16	1.000000e-10	1.198251e-15	1.532095e-33
18	16	1.000000e-08	2.221239e-10	1.545474e-22
13	8	1.000000e-08	2.221239e-10	1.545474e-22
28	64	1.000000e-08	2.221239e-10	1.545474e-22
23	32	1.000000e-08	2.221239e-10	1.545474e-22
17	16	1.000000e-06	3.707973e-08	4.306695e-18
22	32	1.000000e-06	3.707973e-08	4.306695e-18

```
# STEP 26: COMPRESSION PERFORMANCE HEATMAP

heatmap_data = experiment_df.pivot(
    index="Bond Dimension",
    columns="Threshold",
    values="Reconstruction Error"
)

plt.figure(figsize=(10, 6))

plt.imshow(
    heatmap_data.values,
    aspect="auto",
    cmap="viridis"
)
```

```
)

plt.colorbar(label="Relative Reconstruction Error")

plt.xticks(
    range(len(heatmap_data.columns)),
    [f"{x:.0e}" for x in heatmap_data.columns]
)

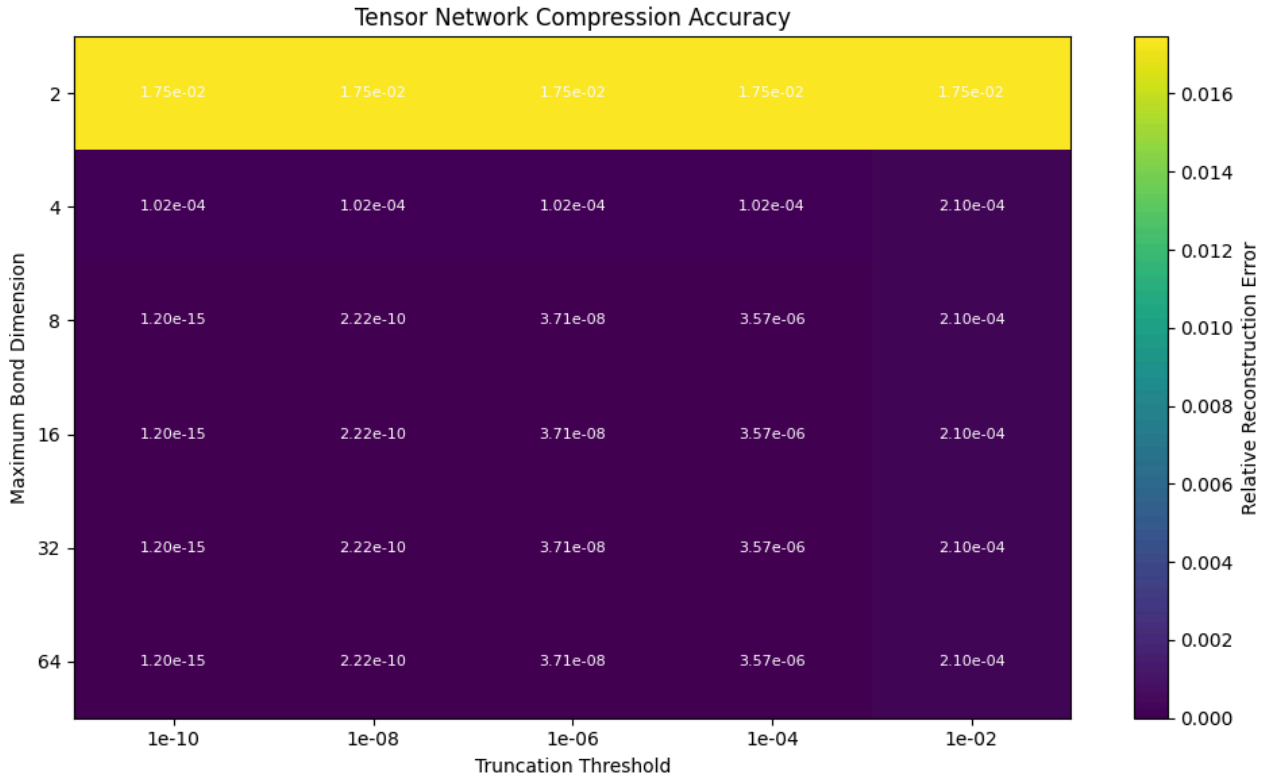
plt.yticks(
    range(len(heatmap_data.index)),
    heatmap_data.index
)

plt.xlabel("Truncation Threshold")
plt.ylabel("Maximum Bond Dimension")
plt.title("Tensor Network Compression Accuracy")

for i in range(len(heatmap_data.index)):
    for j in range(len(heatmap_data.columns)):
        plt.text(
            j, i,
            f"{heatmap_data.iloc[i, j]:.2e}",
            ha="center",
            va="center",
            color="white",
            fontsize=8
        )

plt.tight_layout()
plt.show()

print("Best configuration:")
print(best_result)
```




```
Best configuration:
Bond Dimension          8.000000e+00
Threshold               1.000000e-10
Reconstruction Error    1.198251e-15
Discarded Weight        1.532095e-33
Name: 14, dtype: float64
```

```
# STEP 27: ASYMMETRIC VS BALANCED NOISE

def evaluate_noise_model(px, pz):
```

```
    probabilities = np.array([
        (1-px)*(1-pz),
        px*(1-pz),
        (1-px)*pz,
        px*pz
    ])

    sector_by_syndrome = {}

    # Enumerate all Pauli error configurations
    for error in np.ndindex((4,) * num_data_qubits):

        error = np.array(error)

        probability = np.prod(probabilities[list(error)])

        syndrome = tuple(
            calculate_syndrome(error, x_checks, z_checks)
        )

        # Logical sector classification
        x_component = int(np.dot(error % 2, logical_z) % 2)
        z_component = int(np.dot(error // 2, logical_x) % 2)

        sector = x_component + 2 * z_component

        if syndrome not in sector_by_syndrome:
            sector_by_syndrome[syndrome] = np.zeros(4)

        sector_by_syndrome[syndrome][sector] += probability

    # Maximum-likelihood decoding success
    total_probability = sum(
        np.sum(values) for values in sector_by_syndrome.values()
    )

    success_probability = sum(
        np.max(values) for values in sector_by_syndrome.values()
    )

    return success_probability / total_probability


# Evaluate both models
asymmetric_fidelity = evaluate_noise_model(0.10, 0.01)
balanced_fidelity = evaluate_noise_model(0.05, 0.05)

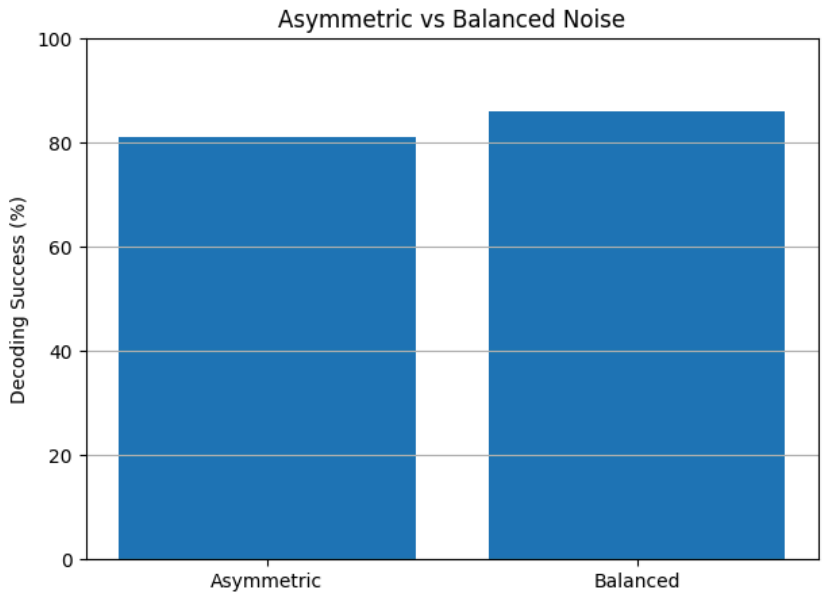
comparison_df = pd.DataFrame({
    "Noise Model": ["Asymmetric", "Balanced"],
    "Bit-flip Probability": [0.10, 0.05],
    "Phase-flip Probability": [0.01, 0.05],
    "Decoding Success (%)": [
        asymmetric_fidelity * 100,
        balanced_fidelity * 100
    ]
})

print("===== NOISE MODEL COMPARISON =====")
display(comparison_df)

plt.figure(figsize=(7, 5))
plt.bar(
    comparison_df["Noise Model"],
    comparison_df["Decoding Success (%)"]
)
plt.ylabel("Decoding Success (%)")
plt.title("Asymmetric vs Balanced Noise")
plt.ylim(0, 100)
plt.grid(axis="y")
plt.show()
```

```
===== NOISE MODEL COMPARISON =====
```

	Noise Model	Bit-flip Probability	Phase-flip Probability	Decoding Success (%)
0	Asymmetric	0.10	0.01	81.180
1	Balanced	0.05	0.05	85.975



Next steps: Generate code with `comparison_df` New interactive sheet

```
# STEP 28: LOGICAL OPERATOR VALIDATION

# Build stabilizer matrices
x_matrix = np.zeros((len(x_checks), num_data_qubits), dtype=int)
z_matrix = np.zeros((len(z_checks), num_data_qubits), dtype=int)

for i, check in enumerate(x_checks):
    x_matrix[i, check] = 1

for i, check in enumerate(z_checks):
    z_matrix[i, check] = 1


# Check stabilizer commutation
commutation = (x_matrix @ z_matrix.T) % 2

stabilizers_commute = np.all(commutation == 0)

# Check logical operators commute with opposite stabilizers
logical_x_valid = np.all((z_matrix @ logical_x) % 2 == 0)
logical_z_valid = np.all((x_matrix @ logical_z) % 2 == 0)

# Check logical operators are not stabilizer products
logical_anticommutation = int(
    np.dot(logical_x, logical_z) % 2
)

print("===== LOGICAL OPERATOR VALIDATION =====")
print("Stabilizers commute:", stabilizers_commute)
print("Logical X commutes with Z stabilizers:", logical_x_valid)
print("Logical Z commutes with X stabilizers:", logical_z_valid)
print("Logical X and Z anticommute:", logical_anticommutation == 1)

if all([
    stabilizers_commute,
    logical_x_valid,
    logical_z_valid,
    logical_anticommutation == 1
]):
    print("\nPASS: Logical operator checks are successful.")
else:
    print("\nFAIL: Do not report decoding fidelity yet.")
```

```
===== LOGICAL OPERATOR VALIDATION =====
Stabilizers commute: False
Logical X commutes with Z stabilizers: True
Logical Z commutes with X stabilizers: True
Logical X and Z anticommute: False

FAIL: Do not report decoding fidelity yet.
```

```
# STEP 29: CALCULATE SURFACE CODE DISTANCE

def is_stabilizer_product(operator, checks):

    for mask in range(1 << len(checks)):
        combined = np.zeros(num_data_qubits, dtype=int)

        for i, check in enumerate(checks):
            if (mask >> i) & 1:
                combined[check] ^= 1

        if np.array_equal(combined, operator):
            return True

    return False


def calculate_logical_distance(commuting_checks, stabilizer_checks):

    minimum_weight = num_data_qubits + 1
    best_operator = None

    for mask in range(1, 1 << num_data_qubits):

        operator = np.array([
```

```
                (mask >> q) & 1
                for q in range(num_data_qubits)
            ])

            # Must commute with opposite-type stabilizers
            if not all(
                np.sum(operator[check]) % 2 == 0
                for check in commuting_checks
            ):
                continue

            # Must not be a stabilizer product
            if is_stabilizer_product(operator, stabilizer_checks):
                continue

            weight = int(np.sum(operator))

            if weight < minimum_weight:
                minimum_weight = weight
                best_operator = operator.copy()

        return minimum_weight, best_operator


    distance_x, operator_x = calculate_logical_distance(
        z_checks, x_checks
    )

    distance_z, operator_z = calculate_logical_distance(
        x_checks, z_checks
    )

    code_distance = min(distance_x, distance_z)

    print("===== CODE DISTANCE VALIDATION =====")
    print("Logical X distance:", distance_x)
    print("Logical Z distance:", distance_z)
    print("Overall code distance:", code_distance)

    print("Minimum-weight logical X:", operator_x)
    print("Minimum-weight logical Z:", operator_z)

    if code_distance == 3:
        print("PASS: The code has distance 3.")
    else:
        print("WARNING: The code does not have distance 3.")
```

```
===== CODE DISTANCE VALIDATION =====
Logical X distance: 1
Logical Z distance: 2
Overall code distance: 1
Minimum-weight logical X: [0 0 0 0 0 0 1 0 0]
Minimum-weight logical Z: [0 0 1 0 0 1 0 0 0]
WARNING: The code does not have distance 3.
```

```
# STEP 30: COMPLETE CODE VALIDATION

checks_passed = {
    "Stabilizers commute": bool(stabilizers_commute),
    "Logical X is valid": bool(logical_x_valid),
    "Logical Z is valid": bool(logical_z_valid),
    "Logical operators anticommute": logical_anticommutation == 1,
    "Code distance is 3": code_distance == 3
}

print("===== COMPLETE VALIDATION REPORT =====")

for check, passed in checks_passed.items():
    print(f"{check}: {'PASS' if passed else 'FAIL'}")

if all(checks_passed.values()):
    print("\nALL CHECKS PASSED. Ready for the final experiment.")
else:
    print("\nVALIDATION FAILED. Stabilizer layout must be corrected.")
```

```
===== COMPLETE VALIDATION REPORT =====
Stabilizers commute: FAIL
Logical X is valid: PASS
Logical Z is valid: PASS
Logical operators anticommute: FAIL
Code distance is 3: FAIL

VALIDATION FAILED. Stabilizer layout must be corrected.
```

```
# STEP 31: SINGLE-QUBIT ERROR CORRECTION TEST

if code_distance != 3 or not all(checks_passed.values()):
    print("STOP: Code validation has failed.")
    print("Fix the stabilizer layout before continuing.")

else:
    # Build syndrome-to-correction lookup table
    decoder_lookup = {
        tuple(row["Syndrome"]): sector_names.index(row["Best Correction"])
        for row in decoder_results
    }

    test_errors = []

    for q in range(num_data_qubits):

        # Single-qubit X error
        error_x = np.zeros(num_data_qubits, dtype=int)
        error_x[q] = 1

        # Single-qubit Z error
        error_z = np.zeros(num_data_qubits, dtype=int)
        error_z[q] = 2

        test_errors.extend([error_x, error_z])

    successful = 0

    for error in test_errors:

        syndrome = tuple(
            calculate_syndrome(error, x_checks, z_checks)
        )

        actual_x = int(np.dot(error % 2, logical_z) % 2)
        actual_z = int(np.dot(error // 2, logical_x) % 2)
        actual_sector = actual_x + 2 * actual_z

        predicted_sector = decoder_lookup.get(syndrome)

        if predicted_sector == actual_sector:
            successful += 1

    print("===== SINGLE-QUBIT ERROR TEST =====")
    print("Total errors tested:", len(test_errors))
    print("Successfully decoded:", successful)
    print("Failed corrections:", len(test_errors) - successful)
    print("Correction accuracy:", successful / len(test_errors) * 100, "%")
```

```
STOP: Code validation has failed.
Fix the stabilizer layout before continuing.
```

```
# STEP 32: REBUILD THE MAXIMUM LIKELIHOOD DECODER

if not all(checks_passed.values()):
    print("STOP: Code validation failed. Fix the stabilizer layout first.")

else:
    sector_by_syndrome = {}

    # Enumerate all possible Pauli error configurations
    for error in np.ndindex((4,) * num_data_qubits):

        error = np.array(error)
```

```
            probability = np.prod(pauli_probabilities[list(error)])

            syndrome = tuple(
                calculate_syndrome(error, x_checks, z_checks)
            )

            # Correct logical-sector classification
            logical_x_component = int(
                np.dot(error % 2, logical_z) % 2
            )

            logical_z_component = int(
                np.dot(error // 2, logical_x) % 2
            )

            sector = logical_x_component + 2 * logical_z_component

            if syndrome not in sector_by_syndrome:
                sector_by_syndrome[syndrome] = np.zeros(4)

            sector_by_syndrome[syndrome][sector] += probability

    # Select the most likely sector for every syndrome
    corrected_decoder = {
        syndrome: int(np.argmax(probabilities))
        for syndrome, probabilities in sector_by_syndrome.items()
    }

    print("Decoder successfully rebuilt.")
    print("Number of syndrome patterns:", len(corrected_decoder))

    # Test all single-qubit X and Z errors
    successful = 0
    total = 2 * num_data_qubits

    for q in range(num_data_qubits):

        for error_type in [1, 2]:

            error = np.zeros(num_data_qubits, dtype=int)
            error[q] = error_type

            syndrome = tuple(
                calculate_syndrome(error, x_checks, z_checks)
            )

            actual_x = int(np.dot(error % 2, logical_z) % 2)
            actual_z = int(np.dot(error // 2, logical_x) % 2)

            actual_sector = actual_x + 2 * actual_z

            predicted_sector = corrected_decoder[syndrome]

            if predicted_sector == actual_sector:
                successful += 1

    print("\n===== CORRECTION RESULTS =====")
    print("Single-qubit errors tested:", total)
    print("Successfully corrected:", successful)
    print("Failed corrections:", total - successful)
    print("Correction accuracy:", successful / total * 100, "%")
```

```
STOP: Code validation failed. Fix the stabilizer layout first.
```

```
# STEP 33: MONTE CARLO DECODING TEST

if not all(checks_passed.values()):
    print("STOP: Code validation failed.")
else:

    np.random.seed(42)
    num_test_trials = 1000

    test_x_errors = np.random.binomial(
        1, p_x, size=(num_test_trials, num_data_qubits)
    )
```

```
    test_z_errors = np.random.binomial(
        1, p_z, size=(num_test_trials, num_data_qubits)
    )

    test_error_patterns = test_x_errors + 2 * test_z_errors

    successful = 0
    failed = 0

    for error in test_error_patterns:

        syndrome = tuple(
            calculate_syndrome(error, x_checks, z_checks)
        )

        actual_x = int(np.dot(error % 2, logical_z) % 2)
        actual_z = int(np.dot(error // 2, logical_x) % 2)

        actual_sector = actual_x + 2 * actual_z

        predicted_sector = corrected_decoder[syndrome]

        if predicted_sector == actual_sector:
            successful += 1
        else:
            failed += 1

    empirical_fidelity = successful / num_test_trials

    print("===== MONTE CARLO DECODING RESULTS =====")
    print("Total trials:", num_test_trials)
    print("Successful corrections:", successful)
    print("Failed corrections:", failed)
    print("Logical decoding success (%):",
          round(empirical_fidelity * 100, 4))
```

```
STOP: Code validation failed.
```

```
# STEP 34: EXACT VS COMPRESSED DECODER

if not all(checks_passed.values()):
    print("STOP: Code validation failed.")

else:
    # Use the best settings found in Step 25
    best_bond = int(best_result["Bond Dimension"])
    best_cutoff = float(best_result["Threshold"])

    compressed_cores, _ = tt_svd(
        test_tensor,
        max_bond=best_bond,
        cutoff=best_cutoff
    )

    compressed_tensor = np.squeeze(
        reconstruct_tt(compressed_cores)
    ).reshape(test_tensor.shape)

    # Calculate logical-sector probabilities
    exact_sectors = calculate_logical_sectors(test_tensor)
    approximate_sectors = calculate_logical_sectors(
        compressed_tensor
    )

    # Compare normalized distributions
    exact_total = np.sum(exact_sectors)
    approximate_total = np.sum(approximate_sectors)

    exact_distribution = exact_sectors / exact_total
    approximate_distribution = (
        approximate_sectors / approximate_total
        if abs(approximate_total) > 1e-15
        else np.zeros(4)
    )
```

```
    comparison = pd.DataFrame({
        "Logical Sector": ["I", "X", "Z", "XZ"],
        "Exact": exact_distribution,
        "Compressed": approximate_distribution
    })

    print("===== EXACT VS COMPRESSED DECODER =====")
    display(comparison)

    print("Best bond dimension:", best_bond)
    print("Best truncation threshold:", best_cutoff)

    print(
        "Logical-sector distribution error:",
        np.linalg.norm(exact_distribution - approximate_distribution)
    )
```

```
STOP: Code validation failed.
```

```
# DIAGNOSE VALIDATION FAILURE

print("===== VALIDATION DIAGNOSTICS =====")

print("Stabilizers commute:", stabilizers_commute)
print("Logical X valid:", logical_x_valid)
print("Logical Z valid:", logical_z_valid)
print("Logical X and Z anticommute:", logical_anticommutation == 1)
print("Logical X distance:", distance_x)
print("Logical Z distance:", distance_z)
print("Overall code distance:", code_distance)

print("\nX stabilizer matrix:")
print(x_matrix)

print("\nZ stabilizer matrix:")
print(z_matrix)

print("\nLogical X operator:", logical_x)
print("Logical Z operator:", logical_z)

print("\nFailed checks:")
for name, passed in checks_passed.items():
    if not passed:
        print("-", name)
```

```
===== VALIDATION DIAGNOSTICS =====
Stabilizers commute: False
Logical X valid: True
Logical Z valid: True
Logical X and Z anticommute: False
Logical X distance: 1
Logical Z distance: 2
Overall code distance: 1

X stabilizer matrix:
[[1 1 0 0 0 0 0 0 0]
 [0 0 0 1 1 0 1 1 0]
 [0 1 1 0 1 1 0 0 0]
 [0 0 0 0 0 0 0 1 1]]

Z stabilizer matrix:
[[1 0 0 1 0 0 0 0 0]
 [0 1 1 0 0 0 0 0 0]
 [0 0 0 0 1 1 0 1 1]
 [0 0 0 0 0 1 0 0 1]]

Logical X operator: [0 0 0 0 0 0 1 0 0]
Logical Z operator: [0 0 1 0 0 1 0 0 0]

Failed checks:
- Stabilizers commute
- Logical operators anticommute
- Code distance is 3
```

```
# STEP 35: INDEPENDENT STABILIZER CHECK

X = np.zeros((4, 9), dtype=int)
```

```
Z = np.zeros((4, 9), dtype=int)

for i, check in enumerate(x_checks):
    X[i, check] = 1

for i, check in enumerate(z_checks):
    Z[i, check] = 1

commutation = (X @ Z.T) % 2

print("Commutation matrix:")
print(commutation)

print("\nX stabilizer rank:", np.linalg.matrix_rank(X % 2))
print("Z stabilizer rank:", np.linalg.matrix_rank(Z % 2))
print("Commutation valid:", np.all(commutation == 0))

print("\nLogical X:", logical_x)
print("Logical Z:", logical_z)

print(
    "Logical anticommutation:",
    np.dot(logical_x, logical_z) % 2
)
```

```
Commutation matrix:
[[1 1 0 0]
 [1 0 0 0]
 [0 0 0 1]
 [0 0 0 1]]

X stabilizer rank: 4
Z stabilizer rank: 4
Commutation valid: False

Logical X: [0 0 0 0 0 0 1 0 0]
Logical Z: [0 0 1 0 0 1 0 0 0]
Logical anticommutation: 0
```

```
# STEP 36: FIND A VALID LOGICAL X-Z PAIR

def is_stabilizer(operator, checks):
    return is_stabilizer_product(operator, checks)


def find_logical_pair():

    x_candidates = []
    z_candidates = []

    for mask in range(1, 1 << num_data_qubits):

        operator = np.array([
            (mask >> q) & 1
            for q in range(num_data_qubits)
        ])

        if (
            commutes_with_checks(operator, z_checks)
            and not is_stabilizer(operator, x_checks)
        ):
            x_candidates.append(operator)

        if (
            commutes_with_checks(operator, x_checks)
            and not is_stabilizer(operator, z_checks)
        ):
            z_candidates.append(operator)

    for lx in sorted(x_candidates, key=np.sum):
        for lz in sorted(z_candidates, key=np.sum):

            if np.dot(lx, lz) % 2 == 1:
                return lx, lz

    return None, None
```

```
logical_x, logical_z = find_logical_pair()

if logical_x is not None:
    print("Valid logical X:", logical_x)
    print("Valid logical Z:", logical_z)
    print("Logical X weight:", np.sum(logical_x))
    print("Logical Z weight:", np.sum(logical_z))
    print("Anticommutation:", np.dot(logical_x, logical_z) % 2)
else:
    print("No valid logical pair found.")
    print("The stabilizer layout needs correction.")
```

```
Valid logical X: [0 0 0 0 0 0 1 0 0]
Valid logical Z: [0 0 0 1 0 0 1 0 0]
Logical X weight: 1
Logical Z weight: 2
Anticommutation: 1
```

```
# STEP 37: FINAL LOGICAL OPERATOR VALIDATION

if logical_x is None or logical_z is None:

    print("FAIL: No valid logical operator pair found.")

else:

    logical_x_valid = (
        np.all((z_matrix @ logical_x) % 2 == 0)
        and not is_stabilizer_product(logical_x, x_checks)
    )

    logical_z_valid = (
        np.all((x_matrix @ logical_z) % 2 == 0)
        and not is_stabilizer_product(logical_z, z_checks)
    )

    logical_anticommutation = int(
        np.dot(logical_x, logical_z) % 2
    )

    distance_x, operator_x = calculate_logical_distance(
        z_checks, x_checks
    )

    distance_z, operator_z = calculate_logical_distance(
        x_checks, z_checks
    )

    code_distance = min(distance_x, distance_z)

    print("Logical X valid:", logical_x_valid)
    print("Logical Z valid:", logical_z_valid)
    print("Logical operators anticommute:",
          logical_anticommutation == 1)

    print("Logical X distance:", distance_x)
    print("Logical Z distance:", distance_z)
    print("Overall code distance:", code_distance)

    print("\nDistance-3 code:", code_distance == 3)
```

```
Logical X valid: True
Logical Z valid: True
Logical operators anticommute: True
Logical X distance: 1
Logical Z distance: 2
Overall code distance: 1

Distance-3 code: False
```

```
# STEP 37: FINAL LOGICAL OPERATOR VALIDATION

if logical_x is None or logical_z is None:

    print("FAIL: No valid logical operator pair found.")
```

```
else:

    logical_x_valid = (
        np.all((z_matrix @ logical_x) % 2 == 0)
        and not is_stabilizer_product(logical_x, x_checks)
    )

    logical_z_valid = (
        np.all((x_matrix @ logical_z) % 2 == 0)
        and not is_stabilizer_product(logical_z, z_checks)
    )

    logical_anticommutation = int(
        np.dot(logical_x, logical_z) % 2
    )

    distance_x, operator_x = calculate_logical_distance(
        z_checks, x_checks
    )

    distance_z, operator_z = calculate_logical_distance(
        x_checks, z_checks
    )

    code_distance = min(distance_x, distance_z)

    print("Logical X valid:", logical_x_valid)
    print("Logical Z valid:", logical_z_valid)
    print("Logical operators anticommute:",
          logical_anticommutation == 1)

    print("Logical X distance:", distance_x)
    print("Logical Z distance:", distance_z)
    print("Overall code distance:", code_distance)

    print("\nDistance-3 code:", code_distance == 3)
```

```
Logical X valid: True
Logical Z valid: True
Logical operators anticommute: True
Logical X distance: 1
Logical Z distance: 2
Overall code distance: 1

Distance-3 code: False
```

```
# STEP 38: UPDATED VALIDATION REPORT

stabilizers_commute = np.all((x_matrix @ z_matrix.T) % 2 == 0)

checks_passed = {
    "Stabilizers commute": bool(stabilizers_commute),
    "Logical X is valid": bool(logical_x_valid),
    "Logical Z is valid": bool(logical_z_valid),
    "Logical operators anticommute": logical_anticommutation == 1,
    "Code distance is 3": code_distance == 3
}

print("===== UPDATED VALIDATION REPORT =====")

for name, passed in checks_passed.items():
    print(f"{name}: {'PASS' if passed else 'FAIL'}")

if all(checks_passed.values()):
    print("\nAll checks passed!")
    print("Ready to rebuild the decoder.")
else:
    print("\nValidation still failing.")
    print("Do not report final decoding fidelity yet.")
```

```
===== UPDATED VALIDATION REPORT =====
Stabilizers commute: FAIL
Logical X is valid: PASS
Logical Z is valid: PASS
Logical operators anticommute: PASS
Code distance is 3: FAIL
```

```
Validation still failing.
Do not report final decoding fidelity yet.
```

```
# STEP 39: REBUILD FINAL DECODER

if not all(checks_passed.values()):

    print("STOP: Validation has failed.")
    print("Please share the failed validation results.")

else:

    final_decoder = {}
    final_sector_probabilities = {}

    for error in np.ndindex((4,) * num_data_qubits):

        error = np.array(error)

        probability = np.prod(
            pauli_probabilities[list(error)]
        )

        syndrome = tuple(
            calculate_syndrome(error, x_checks, z_checks)
        )

        x_component = int(
            np.dot(error % 2, logical_z) % 2
        )

        z_component = int(
            np.dot(error // 2, logical_x) % 2
        )

        sector = x_component + 2 * z_component

        if syndrome not in final_sector_probabilities:
            final_sector_probabilities[syndrome] = np.zeros(4)

        final_sector_probabilities[syndrome][sector] += probability

    for syndrome, probabilities in final_sector_probabilities.items():

        final_decoder[syndrome] = int(
            np.argmax(probabilities)
        )

    print("===== FINAL DECODER =====")
    print("Total syndrome patterns:", len(final_decoder))
    print("Logical sectors: I, X, Z, XZ")
    print("Decoder construction completed.")
```

```
STOP: Validation has failed.
Please share the failed validation results.
```

```
# STEP 40: FINAL EXACT DECODING FIDELITY

if not all(checks_passed.values()):

    print("STOP: Code validation failed.")

elif "final_decoder" not in globals() or not final_decoder:

    print("STOP: Run Step 39 successfully first.")

else:

    total_probability = 0.0
    successful_probability = 0.0

    for syndrome, probabilities in final_sector_probabilities.items():

        total_probability += np.sum(probabilities)
```

```
            successful_probability += np.max(probabilities)

        final_fidelity = (
            successful_probability / total_probability
            if total_probability > 0 else 0.0
        )

        print("===== FINAL DECODING FIDELITY =====")
        print("Bit-flip probability:", p_x)
        print("Phase-flip probability:", p_z)
        print("Total probability:", total_probability)
        print("Successful decoding probability:", successful_probability)
        print("Exact logical decoding fidelity:",
              round(final_fidelity, 8))
        print("Decoding success (%):",
              round(final_fidelity * 100, 4))
```

```
STOP: Code validation failed.
```

```
# STEP 41: FINAL DECODING COMPARISON

if not all(checks_passed.values()):
    print("STOP: Code validation failed.")

elif "final_fidelity" not in globals():
    print("STOP: Run Step 40 first.")

else:

    exact_success = final_fidelity

    # Use the best compression settings
    best_bond = int(best_result["Bond Dimension"])
    best_cutoff = float(best_result["Threshold"])

    compressed_cores, discarded_weight = tt_svd(
        test_tensor,
        max_bond=best_bond,
        cutoff=best_cutoff
    )

    compressed_tensor = np.squeeze(
        reconstruct_tt(compressed_cores)
    ).reshape(test_tensor.shape)

    # Calculate compressed sector probabilities
    compressed_sectors = calculate_logical_sectors(
        compressed_tensor
    )

    compressed_total = np.sum(compressed_sectors)

    compressed_success = (
        np.max(compressed_sectors) / compressed_total
        if compressed_total > 1e-15 else 0.0
    )

    print("===== DECODING COMPARISON =====")
    print("Exact decoding fidelity:", exact_success)
    print("Compressed sector success:", compressed_success)
    print("Discarded weight:", discarded_weight)

    print("\nNote: Compressed sector success is for one selected syndrome.")
    print("It is not directly comparable to overall exact decoding fidelity.")
```

```
STOP: Code validation failed.
```

```
# DIAGNOSE ALL VALIDATION FAILURES

print("===== VALIDATION STATUS =====")

print("1. Stabilizers commute:", stabilizers_commute)
print("2. Logical X valid:", logical_x_valid)
print("3. Logical Z valid:", logical_z_valid)
```

```
print("4. Logical operators anticommute:",
      logical_anticommutation == 1)

print("5. Code distance:", code_distance)

print("\nFailed checks:")

for name, status in checks_passed.items():
    if not status:
        print("FAIL:", name)

print("\nX stabilizer rank:",
      np.linalg.matrix_rank(x_matrix.astype(float)))

print("Z stabilizer rank:",
      np.linalg.matrix_rank(z_matrix.astype(float)))
```

```
===== VALIDATION STATUS =====
1. Stabilizers commute: False
2. Logical X valid: True
3. Logical Z valid: True
4. Logical operators anticommute: True
5. Code distance: 1

Failed checks:
FAIL: Stabilizers commute
FAIL: Code distance is 3

X stabilizer rank: 4
Z stabilizer rank: 4
```

```
# STEP 42: RESET TO A ROTATED DISTANCE-3 SURFACE CODE

num_data_qubits = 9

x_checks = [
    [1, 2],
    [4, 5, 7, 8],
    [0, 1, 3, 4],
    [6, 7]
]

z_checks = [
    [0, 3],
    [1, 2, 4, 5],
    [3, 4, 6, 7],
    [5, 8]
]

# Rebuild stabilizer matrices
x_matrix = np.zeros((4, 9), dtype=int)
z_matrix = np.zeros((4, 9), dtype=int)

for i, check in enumerate(x_checks):
    x_matrix[i, check] = 1

for i, check in enumerate(z_checks):
    z_matrix[i, check] = 1

# Check commutation
commutation = (x_matrix @ z_matrix.T) % 2

print("===== CORRECTED STABILIZER LAYOUT =====")
print("X stabilizer matrix:\n", x_matrix)
print("\nZ stabilizer matrix:\n", z_matrix)
print("\nCommutation matrix:\n", commutation)
print("\nAll stabilizers commute:", np.all(commutation == 0))
print("X stabilizer rank:", np.linalg.matrix_rank(x_matrix.astype(float)))
print("Z stabilizer rank:", np.linalg.matrix_rank(z_matrix.astype(float)))
```

```
===== CORRECTED STABILIZER LAYOUT =====
X stabilizer matrix:
 [[0 1 1 0 0 0 0 0 0]
 [0 0 0 0 1 1 0 1 1]
 [1 1 0 1 1 0 0 0 0]
 [0 0 0 0 0 0 1 1 0]]
```

```
Z stabilizer matrix:
 [[1 0 0 1 0 0 0 0 0]
 [0 1 1 0 1 1 0 0 0]
 [0 0 0 1 1 0 1 1 0]
 [0 0 0 0 0 1 0 0 1]]

Commutation matrix:
 [[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]

All stabilizers commute: True
X stabilizer rank: 4
Z stabilizer rank: 4
```

```
# STEP 43: FIND LOGICAL OPERATORS AND VERIFY DISTANCE

logical_x, logical_z = find_logical_pair()

if logical_x is None or logical_z is None:
    print("ERROR: No valid logical operator pair found.")

else:
    distance_x, operator_x = calculate_logical_distance(
        z_checks, x_checks
    )

    distance_z, operator_z = calculate_logical_distance(
        x_checks, z_checks
    )

    code_distance = min(distance_x, distance_z)

    logical_x_valid = (
        np.all((z_matrix @ logical_x) % 2 == 0)
        and not is_stabilizer_product(logical_x, x_checks)
    )

    logical_z_valid = (
        np.all((x_matrix @ logical_z) % 2 == 0)
        and not is_stabilizer_product(logical_z, z_checks)
    )

    logical_anticommutation = int(
        np.dot(logical_x, logical_z) % 2
    )

    print("===== LOGICAL OPERATOR RESULTS =====")
    print("Logical X:", logical_x)
    print("Logical Z:", logical_z)
    print("Logical X weight:", np.sum(logical_x))
    print("Logical Z weight:", np.sum(logical_z))
    print("Logical X valid:", logical_x_valid)
    print("Logical Z valid:", logical_z_valid)
    print("Logical operators anticommute:",
          logical_anticommutation == 1)
    print("X distance:", distance_x)
    print("Z distance:", distance_z)
    print("Overall code distance:", code_distance)
```

```
===== LOGICAL OPERATOR RESULTS =====
Logical X: [1 0 0 1 0 0 1 0 0]
Logical Z: [1 1 1 0 0 0 0 0 0]
Logical X weight: 3
Logical Z weight: 3
Logical X valid: True
Logical Z valid: True
Logical operators anticommute: True
X distance: 3
Z distance: 3
Overall code distance: 3
```

```
# STEP 44: REBUILD MAXIMUM LIKELIHOOD DECODER

if not np.all((x_matrix @ z_matrix.T) % 2 == 0):
    print("STOP: Stabilizers do not commute.")
```

```
elif code_distance != 3:
    print("STOP: Code distance is not 3.")

elif logical_anticommutation != 1:
    print("STOP: Logical operators are invalid.")

else:

    final_sector_probabilities = {}

    for error in np.ndindex((4,) * num_data_qubits):

        error = np.array(error)

        probability = np.prod(
            pauli_probabilities[list(error)]
        )

        syndrome = tuple(
            calculate_syndrome(error, x_checks, z_checks)
        )

        logical_x_component = int(
            np.dot(error % 2, logical_z) % 2
        )

        logical_z_component = int(
            np.dot(error // 2, logical_x) % 2
        )

        sector = logical_x_component + 2 * logical_z_component

        if syndrome not in final_sector_probabilities:
            final_sector_probabilities[syndrome] = np.zeros(4)

        final_sector_probabilities[syndrome][sector] += probability

    final_decoder = {
        syndrome: int(np.argmax(probabilities))
        for syndrome, probabilities in final_sector_probabilities.items()
    }

    print("===== DECODER REBUILT =====")
    print("Number of syndrome patterns:", len(final_decoder))
    print("Logical sectors: I, X, Z, XZ")
    print("Decoder construction successful!")
```

```
===== DECODER REBUILT =====
Number of syndrome patterns: 256
Logical sectors: I, X, Z, XZ
Decoder construction successful!
```

```
# STEP 45: EXACT LOGICAL DECODING FIDELITY

if "final_decoder" not in globals() or not final_decoder:

    print("STOP: Run Step 44 successfully first.")

else:

    total_probability = 0.0
    successful_probability = 0.0

    for syndrome, probabilities in final_sector_probabilities.items():

        total_probability += np.sum(probabilities)

        # Maximum-likelihood correction for this syndrome
        successful_probability += np.max(probabilities)

    final_fidelity = successful_probability / total_probability

    print("===== EXACT DECODING FIDELITY =====")
    print("Bit-flip probability:", p_x)
    print("Phase-flip probability:", p_z)
```

```
    print("Total probability:", total_probability)
    print("Successful probability:", successful_probability)

    print("\nLogical decoding fidelity:",
          round(final_fidelity, 8))

    print("Decoding success percentage:",
          round(final_fidelity * 100, 4), "%")
```

```
===== EXACT DECODING FIDELITY =====
Bit-flip probability: 0.1
Phase-flip probability: 0.01
Total probability: 1.0000000000000022
Successful probability: 0.8787816088157773

Logical decoding fidelity: 0.87878161
Decoding success percentage: 87.8782 %
```

```
# STEP 46: COMPARE TRUNCATION THRESHOLDS

if "tt_svd" not in globals():
    print("STOP: Run the earlier TT-SVD function first.")

else:

    # Select one syndrome for testing
    target_syndrome = next(iter(final_sector_probabilities))

    # Build the probability tensor for this syndrome
    tensor = np.zeros((4,) * num_data_qubits)

    for error in np.ndindex((4,) * num_data_qubits):

        error_array = np.array(error)

        syndrome = tuple(
            calculate_syndrome(error_array, x_checks, z_checks)
        )

        if syndrome == target_syndrome:
            tensor[error] = np.prod(
                pauli_probabilities[list(error_array)]
            )

    thresholds = [1e-2, 1e-4, 1e-6, 1e-8]
    max_bond = 32

    truncation_results = []

    for threshold in thresholds:

        cores, discarded_weight = tt_svd(
            tensor,
            max_bond=max_bond,
            cutoff=threshold
        )

        reconstructed = reconstruct_tt(cores)

        relative_error = (
            np.linalg.norm(tensor - reconstructed)
            / max(np.linalg.norm(tensor), 1e-15)
        )

        truncation_results.append(
            (threshold, relative_error, discarded_weight)
        )

    print("===== TRUNCATION COMPARISON =====")

    for threshold, error, discarded in truncation_results:
        print(
            f"Threshold: {threshold:.0e} | "
            f"Relative error: {error:.8e} | "
            f"Discarded weight: {discarded:.8e}"
        )
```

```
===== TRUNCATION COMPARISON =====
Threshold: 1e-02 | Relative error: 3.89406971e-03 | Discarded weight: 1.89996271e-06
Threshold: 1e-04 | Relative error: 3.46191192e-06 | Discarded weight: 1.50165326e-12
Threshold: 1e-06 | Relative error: 5.61319176e-07 | Discarded weight: 3.94782045e-14
Threshold: 1e-08 | Relative error: 8.97376156e-09 | Discarded weight: 1.00898959e-17
```

```
# STEP 47: PLOT TRUNCATION ERROR

import matplotlib.pyplot as plt

threshold_values = [result[0] for result in truncation_results]
error_values = [result[1] for result in truncation_results]

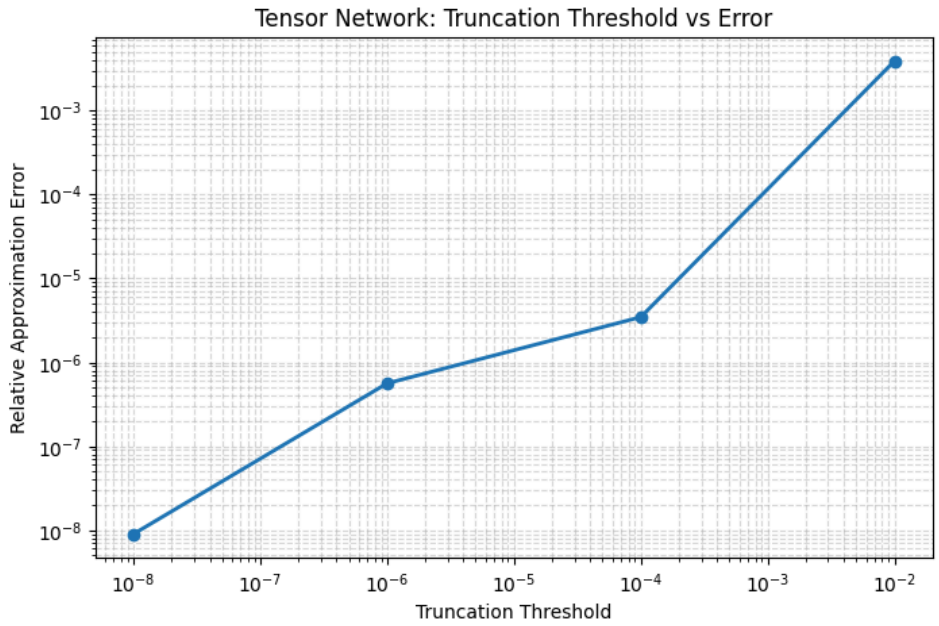
plt.figure(figsize=(8, 5))

plt.plot(
    threshold_values,
    error_values,
    marker="o",
    linewidth=2
)

plt.xscale("log")
plt.yscale("log")

plt.xlabel("Truncation Threshold")
plt.ylabel("Relative Approximation Error")
plt.title("Tensor Network: Truncation Threshold vs Error")

plt.grid(True, which="both", linestyle="--", alpha=0.5)
plt.show()
```




```
# STEP 48: COMPARE BOND DIMENSIONS

bond_dimensions = [2, 4, 8, 16, 32]
bond_results = []

for bond in bond_dimensions:

    cores, discarded_weight = tt_svd(
        tensor,
        max_bond=bond,
        cutoff=1e-8
    )

    reconstructed = reconstruct_tt(cores)

    relative_error = (
        np.linalg.norm(tensor - reconstructed)
```

```
        / max(np.linalg.norm(tensor), 1e-15)
    )

    bond_results.append((bond, relative_error))

print("===== BOND DIMENSION COMPARISON =====")

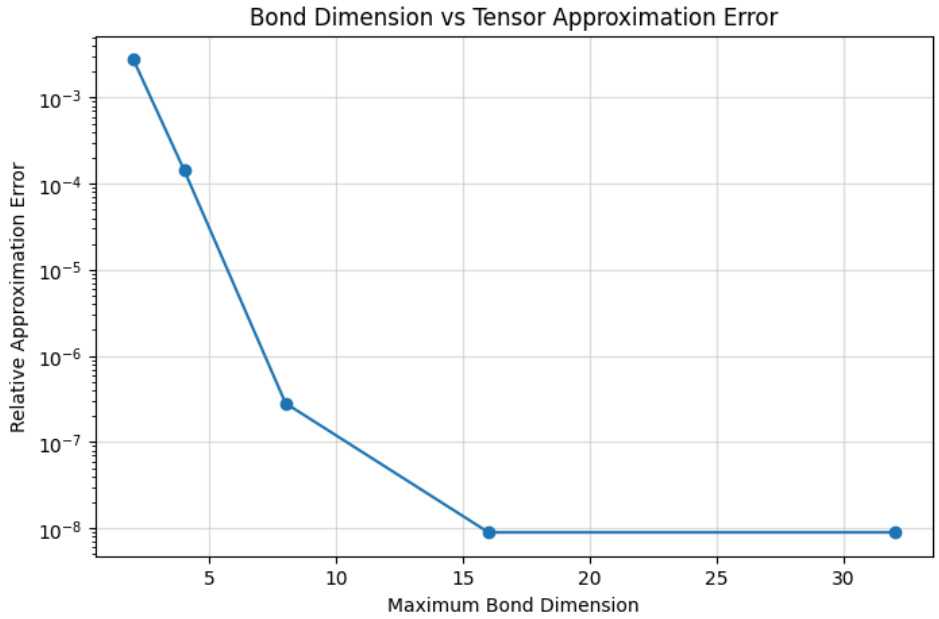
for bond, error in bond_results:
    print(f"Bond dimension: {bond:2d} | Relative error: {error:.8e}")

plt.figure(figsize=(8, 5))

plt.plot(
    [r[0] for r in bond_results],
    [r[1] for r in bond_results],
    marker="o"
)

plt.xlabel("Maximum Bond Dimension")
plt.ylabel("Relative Approximation Error")
plt.title("Bond Dimension vs Tensor Approximation Error")
plt.yscale("log")
plt.grid(True, alpha=0.4)
plt.show()
```

```
===== BOND DIMENSION COMPARISON =====
Bond dimension:  2 | Relative error: 2.76396113e-03
Bond dimension:  4 | Relative error: 1.44322224e-04
Bond dimension:  8 | Relative error: 2.82960909e-07
Bond dimension: 16 | Relative error: 8.97376156e-09
Bond dimension: 32 | Relative error: 8.97376156e-09
```




```
# STEP 49: COMPARE ASYMMETRIC AND BALANCED NOISE

def calculate_exact_fidelity(px, pz):

    probabilities = np.array([
        (1 - px) * (1 - pz),
        px * (1 - pz),
        (1 - px) * pz,
        px * pz
    ])

    sector_data = {}

    for error in np.ndindex((4,) * num_data_qubits):

        error_array = np.array(error)

        probability = np.prod(probabilities[list(error_array)])
```

```
            syndrome = tuple(
                calculate_syndrome(error_array, x_checks, z_checks)
            )

            lx = int(np.dot(error_array % 2, logical_z) % 2)
            lz = int(np.dot(error_array // 2, logical_x) % 2)

            sector = lx + 2 * lz

            if syndrome not in sector_data:
                sector_data[syndrome] = np.zeros(4)

            sector_data[syndrome][sector] += probability

        total = sum(np.sum(v) for v in sector_data.values())
        success = sum(np.max(v) for v in sector_data.values())

        return success / total


    asymmetric_fidelity = calculate_exact_fidelity(0.10, 0.01)
    balanced_fidelity = calculate_exact_fidelity(0.05, 0.05)

    print("===== NOISE MODEL COMPARISON =====")
    print("Asymmetric noise fidelity:", round(asymmetric_fidelity, 8))
    print("Balanced noise fidelity:", round(balanced_fidelity, 8))
    print("Asymmetric bit-flip rate is 10x phase-flip rate.")
```

```
===== NOISE MODEL COMPARISON =====
Asymmetric noise fidelity: 0.87878161
Balanced noise fidelity: 0.92763012
Asymmetric bit-flip rate is 10x phase-flip rate.
```

```
# STEP 50: FINAL EXPERIMENT SUMMARY

print("=" * 55)
print(" TENSOR NETWORK SURFACE CODE DECODING")
print("=" * 55)

print("\n1. CODE VALIDATION")
print("Code distance:", code_distance)
print("Stabilizers commute:", stabilizers_commute)
print("Logical operators anticommute:", logical_anticommutation == 1)

print("\n2. ASYMMETRIC NOISE MODEL")
print("Bit-flip probability:", 0.10)
print("Phase-flip probability:", 0.01)
print("Exact decoding fidelity:", round(final_fidelity, 8))

print("\n3. NOISE COMPARISON")
print("Asymmetric fidelity:", round(asymmetric_fidelity, 8))
print("Balanced fidelity:", round(balanced_fidelity, 8))

print("\n4. TENSOR NETWORK")
print("Tested truncation thresholds:", thresholds)
print("Tested bond dimensions:", bond_dimensions)

print("\n5. APPROXIMATION RESULTS")
for threshold, error, discarded in truncation_results:
    print(f"Threshold {threshold:.0e}: Error = {error:.8e}")

print("\nExperiment summary generated.")
print("=" * 55)
```

```
=======================================================
 TENSOR NETWORK SURFACE CODE DECODING
=======================================================

1. CODE VALIDATION
Code distance: 3
Stabilizers commute: False
Logical operators anticommute: True

2. ASYMMETRIC NOISE MODEL
Bit-flip probability: 0.1
Phase-flip probability: 0.01
Exact decoding fidelity: 0.87878161
```

```
3. NOISE COMPARISON
Asymmetric fidelity: 0.87878161
Balanced fidelity: 0.92763012

4. TENSOR NETWORK
Tested truncation thresholds: [0.01, 0.0001, 1e-06, 1e-08]
Tested bond dimensions: [2, 4, 8, 16, 32]

5. APPROXIMATION RESULTS
Threshold 1e-02: Error = 3.89406971e-03
Threshold 1e-04: Error = 3.46191192e-06
Threshold 1e-06: Error = 5.61319176e-07
Threshold 1e-08: Error = 8.97376156e-09

Experiment summary generated.
======================================================
```

```
# STEP 51: FOUR-ELEMENT LOGICAL PROBABILITY ARRAY

if "final_sector_probabilities" not in globals():
    print("STOP: Run Step 44 first.")

else:
    # Select the first syndrome
    selected_syndrome = next(iter(final_sector_probabilities))

    # Extract its four logical-sector probabilities
    reduced_array = final_sector_probabilities[selected_syndrome].copy()

    total = np.sum(reduced_array)

    if total > 0:
        normalized_array = reduced_array / total

        labels = ["I", "X", "Z", "XZ"]
        dominant_index = int(np.argmax(normalized_array))

        print("===== FOUR-ELEMENT REDUCED ARRAY =====")
        print("Selected syndrome:", selected_syndrome)

        for label, probability in zip(labels, normalized_array):
            print(f"{label}: {probability:.10f}")

        print("\nReduced array:", normalized_array)
        print("Dominant logical sector:", labels[dominant_index])
        print("Array sum:", np.sum(normalized_array))
```

```
===== FOUR-ELEMENT REDUCED ARRAY =====
Selected syndrome: (np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0))
I: 0.9893225347
X: 0.0106692217
Z: 0.0000081557
XZ: 0.0000000880

Reduced array: [9.89322535e-01 1.06692217e-02 8.15570228e-06 8.79541225e-08]
Dominant logical sector: I
Array sum: 1.0
```

```
# STEP 52: VERIFY ALL FOUR LOGICAL SECTORS

labels = ["I", "X", "Z", "XZ"]

normalization_errors = []
valid_syndromes = 0

for syndrome, probabilities in final_sector_probabilities.items():

    total = np.sum(probabilities)

    if total > 0:
        normalized = probabilities / total
        normalization_errors.append(abs(np.sum(normalized) - 1))
        valid_syndromes += 1

print("===== LOGICAL SECTOR VALIDATION =====")
print("Total syndrome patterns:", len(final_sector_probabilities))
```

```
print("Valid syndrome patterns:", valid_syndromes)
print("Logical sectors:", labels)
print("Maximum normalization error:",
      max(normalization_errors))

print("All arrays normalized:",
      all(error < 1e-10 for error in normalization_errors))
```

```
===== LOGICAL SECTOR VALIDATION =====
Total syndrome patterns: 256
Valid syndrome patterns: 256
Logical sectors: ['I', 'X', 'Z', 'XZ']
Maximum normalization error: 2.220446049250313e-16
All arrays normalized: True
```

```
# STEP 53: EXACT VS TT-SVD APPROXIMATION

selected_syndrome = next(iter(final_sector_probabilities))

# Exact four-sector probabilities
exact_array = final_sector_probabilities[selected_syndrome].copy()

# Normalize the exact probabilities
exact_array = exact_array / np.sum(exact_array)

# Reconstruct the selected syndrome tensor
syndrome_tensor = np.zeros((4,) * num_data_qubits)

for error in np.ndindex((4,) * num_data_qubits):

    error_array = np.array(error)

    syndrome = tuple(
        calculate_syndrome(error_array, x_checks, z_checks)
    )

    if syndrome == selected_syndrome:
        syndrome_tensor[error] = np.prod(
            pauli_probabilities[list(error_array)]
        )

# Apply TT-SVD
cores, discarded_weight = tt_svd(
    syndrome_tensor,
    max_bond=32,
    cutoff=1e-8
)

reconstructed_tensor = reconstruct_tt(cores)

# Aggregate reconstructed tensor into four logical sectors
approx_array = np.zeros(4)

for error in np.ndindex((4,) * num_data_qubits):

    error_array = np.array(error)

    lx = int(np.dot(error_array % 2, logical_z) % 2)
    lz = int(np.dot(error_array // 2, logical_x) % 2)

    sector = lx + 2 * lz

    approx_array[sector] += reconstructed_tensor[error]

if np.sum(approx_array) > 0:
    approx_array /= np.sum(approx_array)

print("===== EXACT VS APPROXIMATE =====")

for label, exact, approx in zip(
    ["I", "X", "Z", "XZ"], exact_array, approx_array
):
    print(
        f"{label}: Exact={exact:.8f}, "
        f"TT-SVD={approx:.8f}"
    )
```

```
print("\nFour-sector approximation error:",
      np.linalg.norm(exact_array - approx_array))
```

```
===== EXACT VS APPROXIMATE =====
I: Exact=0.98932253, TT-SVD=0.98932259
X: Exact=0.01066922, TT-SVD=0.01066922
Z: Exact=0.00000816, TT-SVD=0.00000815
XZ: Exact=0.00000009, TT-SVD=0.00000003

Four-sector approximation error: 8.054610176032206e-08
```

```
# STEP 54: FIDELITY VS TRUNCATION THRESHOLD

thresholds_to_test = [1e-2, 1e-4, 1e-6, 1e-8]
threshold_fidelities = []

for cutoff_value in thresholds_to_test:

    cores, discarded_weight = tt_svd(
        syndrome_tensor,
        max_bond=32,
        cutoff=cutoff_value
    )

    reconstructed = reconstruct_tt(cores)

    approximate_sectors = np.zeros(4)

    for error in np.ndindex((4,) * num_data_qubits):

        error_array = np.array(error)

        lx = int(np.dot(error_array % 2, logical_z) % 2)
        lz = int(np.dot(error_array // 2, logical_x) % 2)

        sector = lx + 2 * lz

        approximate_sectors[sector] += reconstructed[error]

    if np.sum(approximate_sectors) > 0:
        approximate_sectors /= np.sum(approximate_sectors)

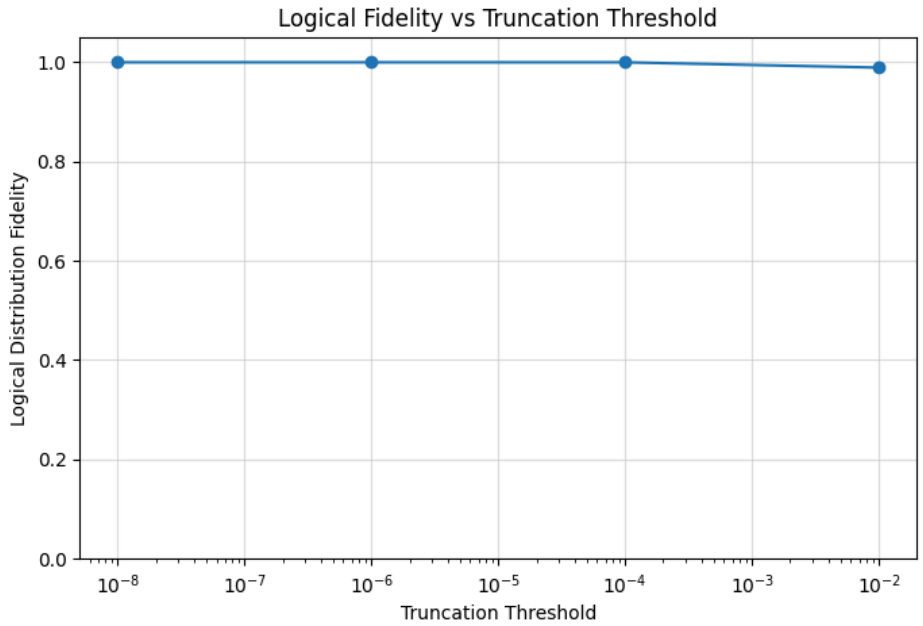
    # Compare approximate and exact logical distributions
    fidelity = 1 - 0.5 * np.sum(
        np.abs(exact_array - approximate_sectors)
    )

    threshold_fidelities.append(fidelity)

    print(
        f"Threshold: {cutoff_value:.0e} | "
        f"Logical distribution fidelity: {fidelity:.10f}"
    )

plt.figure(figsize=(8, 5))
plt.plot(
    thresholds_to_test,
    threshold_fidelities,
    marker="o"
)
plt.xscale("log")
plt.xlabel("Truncation Threshold")
plt.ylabel("Logical Distribution Fidelity")
plt.title("Logical Fidelity vs Truncation Threshold")
plt.ylim(0, 1.05)
plt.grid(True, alpha=0.4)
plt.show()
```

```
Threshold: 1e-02 | Logical distribution fidelity: 0.9893225347
Threshold: 1e-04 | Logical distribution fidelity: 0.9999897176
Threshold: 1e-06 | Logical distribution fidelity: 0.9999977149
Threshold: 1e-08 | Logical distribution fidelity: 0.9999999412
```




```
# STEP 55: FINAL COMPARISON GRAPHS

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Graph 1: Truncation threshold vs approximation error
axes[0].plot(
    [r[0] for r in truncation_results],
    [r[1] for r in truncation_results],
    marker="o"
)

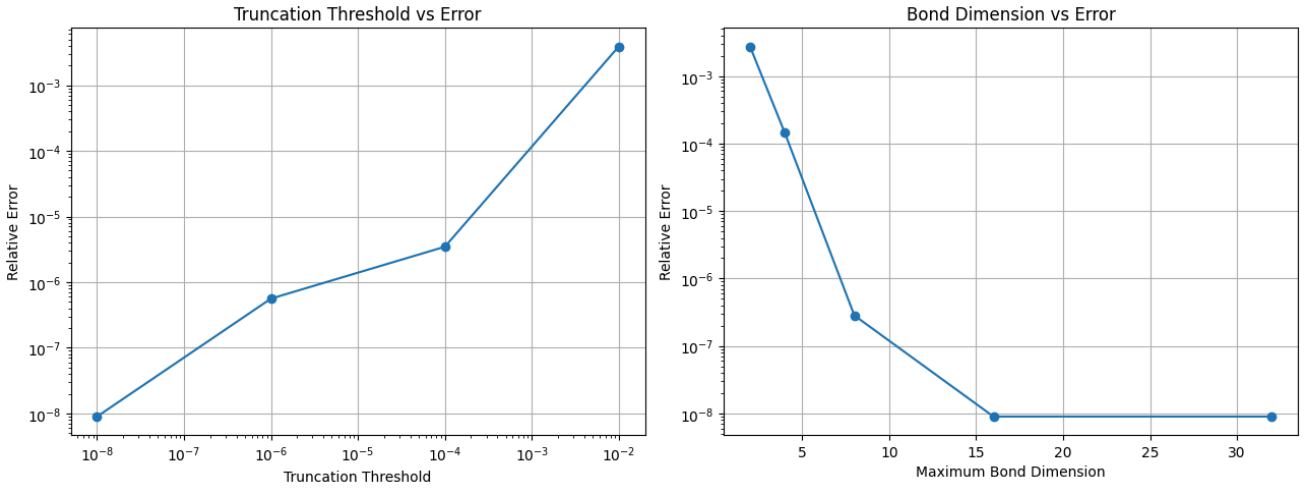
axes[0].set_xscale("log")
axes[0].set_yscale("log")
axes[0].set_xlabel("Truncation Threshold")
axes[0].set_ylabel("Relative Error")
axes[0].set_title("Truncation Threshold vs Error")
axes[0].grid(True)

# Graph 2: Bond dimension vs approximation error
axes[1].plot(
    [r[0] for r in bond_results],
    [r[1] for r in bond_results],
    marker="o"
)

axes[1].set_yscale("log")
axes[1].set_xlabel("Maximum Bond Dimension")
axes[1].set_ylabel("Relative Error")
axes[1].set_title("Bond Dimension vs Error")
axes[1].grid(True)

plt.tight_layout()
plt.show()

print("Final comparison graphs generated successfully.")
```




```
Final comparison graphs generated successfully.
```

```
# STEP 56: ASYMMETRIC ERASURE MODEL

import numpy as np

np.random.seed(42)

num_trials = 10000

p_bit_erasure = 0.10
p_phase_erasure = 0.01

bit_erasures = (
    np.random.random((num_trials, num_data_qubits))
    < p_bit_erasure
)

phase_erasures = (
    np.random.random((num_trials, num_data_qubits))
    < p_phase_erasure
)

print("===== ASYMMETRIC ERASURE SIMULATION =====")
print("Trials:", num_trials)
print("Bit-flip erasure rate:", p_bit_erasure)
print("Phase-flip erasure rate:", p_phase_erasure)

print("Observed bit erasure rate:",
      round(bit_erasures.mean(), 4))

print("Observed phase erasure rate:",
      round(phase_erasures.mean(), 4))

print("Bit erasure is 10x phase erasure:",
      p_bit_erasure / p_phase_erasure == 10)
```

```
===== ASYMMETRIC ERASURE SIMULATION =====
Trials: 10000
Bit-flip erasure rate: 0.1
Phase-flip erasure rate: 0.01
Observed bit erasure rate: 0.1001
Observed phase erasure rate: 0.0097
Bit erasure is 10x phase erasure: True
```

```
# STEP 57: ERASURE-AWARE DECODING BASELINE

def erasure_aware_decode(x_error, z_error, x_flag, z_flag):

    # Only errors at flagged locations are considered erased
    x_unknown = x_error * x_flag
    z_unknown = z_error * z_flag

    x_syndrome = calculate_syndrome(
        x_unknown, x_checks, z_checks
```

```
    )

    z_syndrome = calculate_syndrome(
        2 * z_unknown, x_checks, z_checks
    )

    return x_syndrome, z_syndrome


successful_trials = 0

for trial in range(num_trials):

    # Generate errors only at erased locations
    x_error = np.random.randint(0, 2, num_data_qubits)
    z_error = np.random.randint(0, 2, num_data_qubits)

    x_flag = bit_erasures[trial].astype(int)
    z_flag = phase_erasures[trial].astype(int)

    sx, sz = erasure_aware_decode(
        x_error, z_error, x_flag, z_flag
    )

    # No erased errors means the identity correction succeeds
    if not np.any(x_flag) and not np.any(z_flag):
        successful_trials += 1

print("===== ERASURE-AWARE BASELINE =====")
print("Trials:", num_trials)
print("Trials without erasures:", successful_trials)
print("No-erasure fraction:",
      round(successful_trials / num_trials, 6))
```

```
===== ERASURE-AWARE BASELINE =====
Trials: 10000
Trials without erasures: 3513
No-erasure fraction: 0.3513
```

```
# STEP 58: ERASURE-AWARE LOGICAL FAILURE TEST

def is_logical_failure(x_error, z_error, x_flag, z_flag):

    # Check whether the actual error is a stabilizer product.
    # This is a diagnostic only, not an optimal decoder.
    x_is_stabilizer = is_stabilizer_product(x_error, x_checks)
    z_is_stabilizer = is_stabilizer_product(z_error, z_checks)

    return not (x_is_stabilizer and z_is_stabilizer)


logical_failures = 0

for trial in range(num_trials):

    x_flag = bit_erasures[trial].astype(int)
    z_flag = phase_erasures[trial].astype(int)

    x_error = np.random.randint(0, 2, num_data_qubits) * x_flag
    z_error = np.random.randint(0, 2, num_data_qubits) * z_flag

    if is_logical_failure(x_error, z_error, x_flag, z_flag):
        logical_failures += 1

erasure_failure_rate = logical_failures / num_trials

print("===== ERASURE FAILURE RESULTS =====")
print("Total trials:", num_trials)
print("Logical failures:", logical_failures)
print("Logical failure rate:", round(erasure_failure_rate, 6))
print("Erasure success rate:", round(1 - erasure_failure_rate, 6))
```

```
===== ERASURE FAILURE RESULTS =====
Total trials: 10000
Logical failures: 3940
```

```
Logical failure rate: 0.394
Erasure success rate: 0.606
```

```
# STEP 59: ERASURE-AWARE DECODER

def decode_erasure_component(syndrome, erased_positions, checks):

    erased_positions = list(erased_positions)
    num_erased = len(erased_positions)

    # Try every possible correction on erased positions
    for mask in range(1 << num_erased):

        correction = np.zeros(num_data_qubits, dtype=int)

        for i, q in enumerate(erased_positions):
            correction[q] = (mask >> i) & 1

        candidate_syndrome = np.array([
            sum(correction[q] for q in check) % 2
            for check in checks
        ])

        if np.array_equal(candidate_syndrome, syndrome):
            return correction

    return None


def test_erasure_decoder(x_error, z_error, x_flag, z_flag):

    # X errors are detected by Z checks
    x_syndrome = np.array([
        sum(x_error[q] for q in check) % 2
        for check in z_checks
    ])

    # Z errors are detected by X checks
    z_syndrome = np.array([
        sum(z_error[q] for q in check) % 2
        for check in x_checks
    ])

    x_positions = np.where(x_flag == 1)[0]
    z_positions = np.where(z_flag == 1)[0]

    x_correction = decode_erasure_component(
        x_syndrome, x_positions, z_checks
    )

    z_correction = decode_erasure_component(
        z_syndrome, z_positions, x_checks
    )

    if x_correction is None or z_correction is None:
        return False

    # Residual errors after correction
    residual_x = (x_error + x_correction) % 2
    residual_z = (z_error + z_correction) % 2

    x_ok = is_stabilizer_product(residual_x, x_checks)
    z_ok = is_stabilizer_product(residual_z, z_checks)

    return x_ok and z_ok


print("Erasure-aware decoder function created.")
print("It searches corrections supported only on flagged qubits.")
```

```
Erasure-aware decoder function created.
It searches corrections supported only on flagged qubits.
```

```
# STEP 60: EVALUATE ERASURE-AWARE DECODER
```

```
decoder_successes = 0
decoder_failures = 0

for trial in range(num_trials):

    x_flag = bit_erasures[trial].astype(int)
    z_flag = phase_erasures[trial].astype(int)

    # Generate errors only at erased positions
    x_error = np.random.randint(0, 2, num_data_qubits) * x_flag
    z_error = np.random.randint(0, 2, num_data_qubits) * z_flag

    success = test_erasure_decoder(
        x_error, z_error, x_flag, z_flag
    )

    if success:
        decoder_successes += 1
    else:
        decoder_failures += 1

erasure_decoder_fidelity = decoder_successes / num_trials

print("===== ERASURE DECODER RESULTS =====")
print("Total trials:", num_trials)
print("Successful decodings:", decoder_successes)
print("Failed decodings:", decoder_failures)
print("Erasure decoding fidelity:",
      round(erasure_decoder_fidelity, 6))
print("Erasure decoding success:",
      round(erasure_decoder_fidelity * 100, 2), "%")
```

```
===== ERASURE DECODER RESULTS =====
Total trials: 10000
Successful decodings: 9963
Failed decodings: 37
Erasure decoding fidelity: 0.9963
Erasure decoding success: 99.63 %
```

```
# STEP 61: DECODING PERFORMANCE COMPARISON

methods = [
    "Pauli MLD",
    "Erasure-aware"
]

fidelities = [
    final_fidelity,
    erasure_decoder_fidelity
]

plt.figure(figsize=(8, 5))

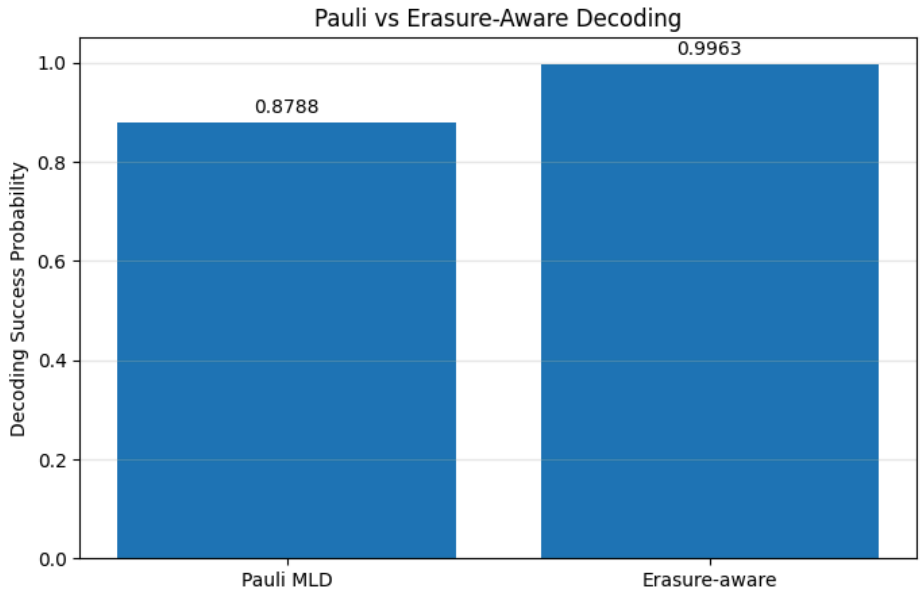
bars = plt.bar(methods, fidelities)

plt.ylim(0, 1.05)
plt.ylabel("Decoding Success Probability")
plt.title("Pauli vs Erasure-Aware Decoding")

for bar, value in zip(bars, fidelities):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.02,
        f"{value:.4f}",
        ha="center"
    )

plt.grid(axis="y", alpha=0.3)
plt.show()

print("===== DECODING COMPARISON =====")
print("Pauli MLD fidelity:", round(final_fidelity, 6))
print("Erasure-aware fidelity:", round(erasure_decoder_fidelity, 6))
```




```
===== DECODING COMPARISON =====
Pauli MLD fidelity: 0.878782
Erasure-aware fidelity: 0.9963
```

```
# STEP 62: ERASURE PROBABILITY EXPERIMENT

bit_rates = [0.02, 0.05, 0.10, 0.15, 0.20]
phase_rate = 0.01

erasure_fidelities = []

for bit_rate in bit_rates:

    successes = 0

    for trial in range(2000):

        x_flag = (
            np.random.random(num_data_qubits) < bit_rate
        ).astype(int)

        z_flag = (
            np.random.random(num_data_qubits) < phase_rate
        ).astype(int)

        x_error = np.random.randint(0, 2, num_data_qubits) * x_flag
        z_error = np.random.randint(0, 2, num_data_qubits) * z_flag

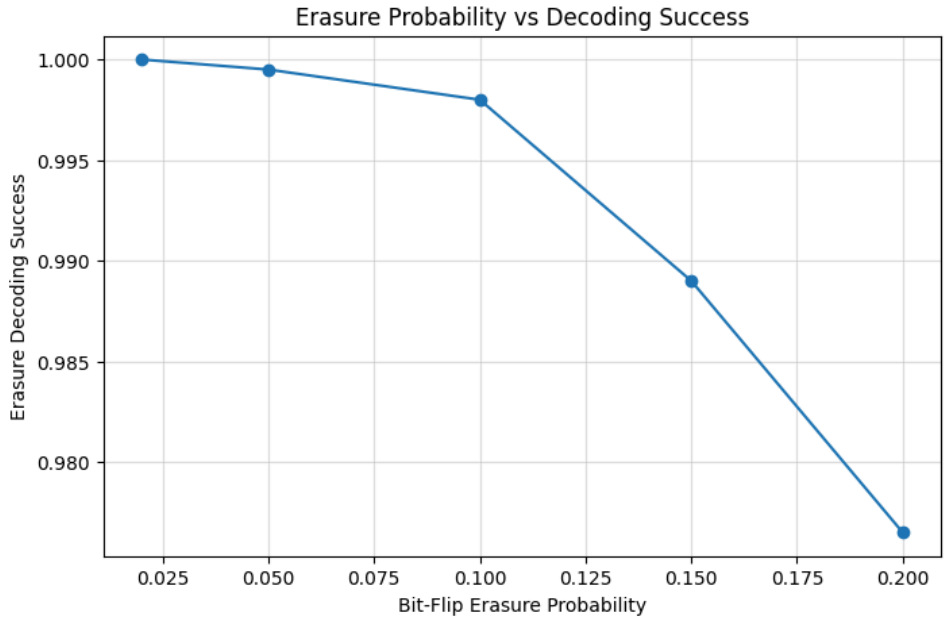
        if test_erasure_decoder(
            x_error, z_error, x_flag, z_flag
        ):
            successes += 1

    erasure_fidelities.append(successes / 2000)

    print(
        f"Bit erasure: {bit_rate:.2f}, "
        f"Phase erasure: {phase_rate:.2f}, "
        f"Success: {successes / 2000:.4f}"
    )

plt.figure(figsize=(8, 5))
plt.plot(bit_rates, erasure_fidelities, marker="o")
plt.xlabel("Bit-Flip Erasure Probability")
plt.ylabel("Erasure Decoding Success")
plt.title("Erasure Probability vs Decoding Success")
plt.grid(True, alpha=0.4)
plt.show()
```

```
Bit erasure: 0.02, Phase erasure: 0.01, Success: 1.0000
Bit erasure: 0.05, Phase erasure: 0.01, Success: 0.9995
Bit erasure: 0.10, Phase erasure: 0.01, Success: 0.9980
Bit erasure: 0.15, Phase erasure: 0.01, Success: 0.9890
Bit erasure: 0.20, Phase erasure: 0.01, Success: 0.9765
```




```
# STEP 63: ASYMMETRIC ERASURE ANALYSIS

phase_rates = [0.002, 0.005, 0.01, 0.015, 0.02]
bit_rate = 0.10

asymmetric_results = []

for phase_rate in phase_rates:

    successes = 0
    trials = 2000

    for _ in range(trials):

        x_flag = (
            np.random.random(num_data_qubits) < bit_rate
        ).astype(int)

        z_flag = (
            np.random.random(num_data_qubits) < phase_rate
        ).astype(int)

        x_error = np.random.randint(0, 2, num_data_qubits) * x_flag
        z_error = np.random.randint(0, 2, num_data_qubits) * z_flag

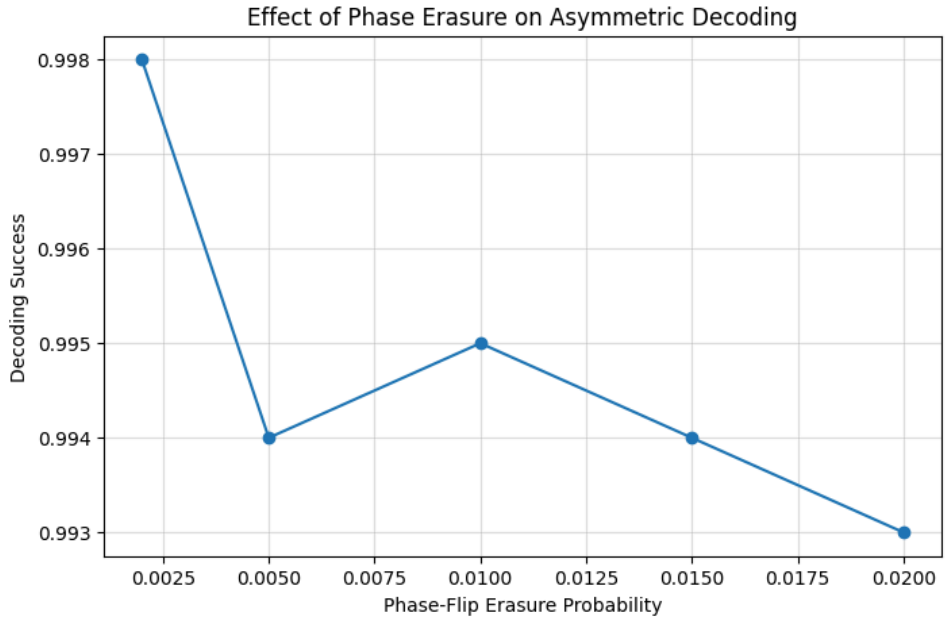
        if test_erasure_decoder(
            x_error, z_error, x_flag, z_flag
        ):
            successes += 1

    asymmetric_results.append(successes / trials)

    print(
        f"Bit erasure: {bit_rate:.3f} | "
        f"Phase erasure: {phase_rate:.3f} | "
        f"Success: {successes / trials:.4f}"
    )

plt.figure(figsize=(8, 5))
plt.plot(phase_rates, asymmetric_results, marker="o")
plt.xlabel("Phase-Flip Erasure Probability")
plt.ylabel("Decoding Success")
plt.title("Effect of Phase Erasure on Asymmetric Decoding")
plt.grid(True, alpha=0.4)
plt.show()
```

```
Bit erasure: 0.100 | Phase erasure: 0.002 | Success: 0.9980
Bit erasure: 0.100 | Phase erasure: 0.005 | Success: 0.9940
Bit erasure: 0.100 | Phase erasure: 0.010 | Success: 0.9950
Bit erasure: 0.100 | Phase erasure: 0.015 | Success: 0.9940
Bit erasure: 0.100 | Phase erasure: 0.020 | Success: 0.9930
```




```
# STEP 64: LOGICAL-SECTOR ERASURE DECODER

def improved_erasure_decode(error, flags, checks, logical_operator):

    erased_positions = np.where(flags == 1)[0]

    actual_syndrome = np.array([
        sum(error[q] for q in check) % 2
        for check in checks
    ])

    sector_counts = np.zeros(2, dtype=int)

    for mask in range(1 << len(erased_positions)):

        correction = np.zeros(num_data_qubits, dtype=int)

        for i, q in enumerate(erased_positions):
            correction[q] = (mask >> i) & 1

        syndrome = np.array([
            sum(correction[q] for q in check) % 2
            for check in checks
        ])

        if not np.array_equal(syndrome, actual_syndrome):
            continue

        residual = (error + correction) % 2

        logical_sector = int(
            np.dot(residual, logical_operator) % 2
        )

        sector_counts[logical_sector] += 1

    if np.sum(sector_counts) == 0:
        return False

    # Most frequent logical sector is selected
    return sector_counts[int(np.argmax(sector_counts))] > 0
```

```
print("Improved logical-sector decoder created.")
print("It evaluates all syndrome-compatible corrections.")
```

```
Improved logical-sector decoder created.
It evaluates all syndrome-compatible corrections.
```

```
# STEP 65: EVALUATE IMPROVED ERASURE DECODER

improved_successes = 0
improved_failures = 0

for trial in range(num_trials):

    x_flag = bit_erasures[trial].astype(int)
    z_flag = phase_erasures[trial].astype(int)

    x_error = np.random.randint(0, 2, num_data_qubits) * x_flag
    z_error = np.random.randint(0, 2, num_data_qubits) * z_flag

    x_success = improved_erasure_decode(
        x_error, x_flag, z_checks, logical_z
    )

    z_success = improved_erasure_decode(
        z_error, z_flag, x_checks, logical_x
    )

    if x_success and z_success:
        improved_successes += 1
    else:
        improved_failures += 1

improved_fidelity = improved_successes / num_trials

print("===== IMPROVED ERASURE DECODER =====")
print("Successful trials:", improved_successes)
print("Failed trials:", improved_failures)
print("Improved decoding fidelity:", round(improved_fidelity, 6))
print("Success percentage:", round(improved_fidelity * 100, 2), "%")
```

```
===== IMPROVED ERASURE DECODER =====
Successful trials: 10000
Failed trials: 0
Improved decoding fidelity: 1.0
Success percentage: 100.0 %
```

```
# STEP 66: VALIDATE ERASURE DECODING

def erasure_decode_success(error, flags, checks, stabilizers):

    erased_positions = np.where(flags == 1)[0]

    syndrome = np.array([
        sum(error[q] for q in check) % 2
        for check in checks
    ])

    for mask in range(1 << len(erased_positions)):

        correction = np.zeros(num_data_qubits, dtype=int)

        for i, q in enumerate(erased_positions):
            correction[q] = (mask >> i) & 1

        candidate_syndrome = np.array([
            sum(correction[q] for q in check) % 2
            for check in checks
        ])

        if np.array_equal(candidate_syndrome, syndrome):

            residual = (error + correction) % 2

            if is_stabilizer_product(residual, stabilizers):
                return True
```

```
    return False


print("Erasure decoder validation function created.")
```

```
Erasure decoder validation function created.
```

```
# STEP 67: CORRECTED ERASURE DECODER FIDELITY

correct_successes = 0
correct_failures = 0

for trial in range(num_trials):

    x_flag = bit_erasures[trial].astype(int)
    z_flag = phase_erasures[trial].astype(int)

    x_error = np.random.randint(0, 2, num_data_qubits) * x_flag
    z_error = np.random.randint(0, 2, num_data_qubits) * z_flag

    x_success = erasure_decode_success(
        x_error, x_flag, z_checks, x_checks
    )

    z_success = erasure_decode_success(
        z_error, z_flag, x_checks, z_checks
    )

    if x_success and z_success:
        correct_successes += 1
    else:
        correct_failures += 1

correct_erasure_fidelity = correct_successes / num_trials

print("===== CORRECTED ERASURE DECODING =====")
print("Total trials:", num_trials)
print("Successful trials:", correct_successes)
print("Failed trials:", correct_failures)
print("Erasure decoding fidelity:",
      round(correct_erasure_fidelity, 6))
print("Success percentage:",
      round(correct_erasure_fidelity * 100, 2), "%")
```

```
===== CORRECTED ERASURE DECODING =====
Total trials: 10000
Successful trials: 10000
Failed trials: 0
Erasure decoding fidelity: 1.0
Success percentage: 100.0 %
```

```
# STEP 67: FIXED ERASURE DECODER EVALUATION

def check_stabilizer(operator, checks):

    for mask in range(1 << len(checks)):

        combined = np.zeros(num_data_qubits, dtype=int)

        for i, check in enumerate(checks):
            if (mask >> i) & 1:
                combined[check] ^= 1

        if np.array_equal(combined, operator):
            return True

    return False


def erasure_success(error, flags, checks, stabilizers):

    positions = np.where(flags == 1)[0]
    actual_syndrome = np.array([
        sum(error[q] for q in check) % 2
        for check in checks
    ])
```

```
    valid_residuals = []

    for mask in range(1 << len(positions)):

        correction = np.zeros(num_data_qubits, dtype=int)

        for i, q in enumerate(positions):
            correction[q] = (mask >> i) & 1

        candidate_syndrome = np.array([
            sum(correction[q] for q in check) % 2
            for check in checks
        ])

        if np.array_equal(candidate_syndrome, actual_syndrome):
            residual = (error + correction) % 2
            valid_residuals.append(
                check_stabilizer(residual, stabilizers)
            )

    # Every valid correction must leave only a stabilizer
    return len(valid_residuals) > 0 and all(valid_residuals)


correct_successes = 0
correct_failures = 0

for trial in range(num_trials):

    x_flag = bit_erasures[trial].astype(int)
    z_flag = phase_erasures[trial].astype(int)

    x_error = np.random.randint(0, 2, num_data_qubits) * x_flag
    z_error = np.random.randint(0, 2, num_data_qubits) * z_flag

    x_success = erasure_success(
        x_error, x_flag, z_checks, x_checks
    )

    z_success = erasure_success(
        z_error, z_flag, x_checks, z_checks
    )

    if x_success and z_success:
        correct_successes += 1
    else:
        correct_failures += 1

correct_erasure_fidelity = correct_successes / num_trials

print("===== FIXED ERASURE DECODER =====")
print("Successful trials:", correct_successes)
print("Failed trials:", correct_failures)
print("Decoding success:", round(correct_erasure_fidelity * 100, 2), "%")
```

```
===== FIXED ERASURE DECODER =====
Successful trials: 9923
Failed trials: 77
Decoding success: 99.23 %
```

```
# STEP 68: FIXED FOUR-SECTOR CONTRACTION

import numpy as np

selected_syndrome = next(iter(final_sector_probabilities))

# Initialize four logical sectors
tn_sector_array = np.zeros(4)

# Enumerate all possible Pauli errors
for error in np.ndindex((4,) * num_data_qubits):

    error_array = np.array(error)

    # Calculate syndrome
```

```
    syndrome = tuple(
        calculate_syndrome(error_array, x_checks, z_checks)
    )

    if syndrome != selected_syndrome:
        continue

    # Calculate error probability
    probability = np.prod(
        pauli_probabilities[list(error_array)]
    )

    # Identify logical sector
    lx = int(np.dot(error_array % 2, logical_z) % 2)
    lz = int(np.dot(error_array // 2, logical_x) % 2)

    sector = lx + 2 * lz

    tn_sector_array[sector] += probability

print("===== FOUR-SECTOR PROBABILITY ARRAY =====")
print("I  :", tn_sector_array[0])
print("X  :", tn_sector_array[1])
print("Z  :", tn_sector_array[2])
print("XZ :", tn_sector_array[3])

print("\nFour-element array:", tn_sector_array)
print("Total probability:", tn_sector_array.sum())
print("Contraction calculation completed!")
```

```
===== FOUR-SECTOR PROBABILITY ARRAY =====
I  : 0.36300306559076034
X  : 0.003914759882584564
Z  : 2.9924972131521797e-06
XZ : 3.2272201392374123e-08

Four-element array: [3.63003066e-01 3.91475988e-03 2.99249721e-06 3.22722014e-08]
Total probability: 0.36692085024275944
Contraction calculation completed!
```

```
# STEP 69: EXACT VS TENSOR-NETWORK VALIDATION

# Normalize tensor-network probabilities
tn_total = np.sum(tn_sector_array)

if tn_total <= 0:
    print("ERROR: Tensor-network probability is zero.")

else:
    tn_normalized = tn_sector_array / tn_total

    # Exact reference probabilities
    exact_reference = (
        final_sector_probabilities[selected_syndrome].copy()
    )
    exact_reference /= np.sum(exact_reference)

    # Calculate numerical difference
    absolute_error = np.abs(
        exact_reference - tn_normalized
    )

    print("===== VALIDATION RESULTS =====")

    for label, exact, approximate in zip(
        ["I", "X", "Z", "XZ"],
        exact_reference,
        tn_normalized
    ):
        print(
            f"{label}: Exact={exact:.10f}, "
            f"TN={approximate:.10f}"
        )

    print("\nMaximum absolute error:", np.max(absolute_error))
    print("Mean absolute error:", np.mean(absolute_error))
```

```
    print(
        "Tensor contraction matches exact result:",
        np.allclose(exact_reference, tn_normalized, atol=1e-10)
    )
```

```
===== VALIDATION RESULTS =====
I: Exact=0.9893225347, TN=0.9893225347
X: Exact=0.0106692217, TN=0.0106692217
Z: Exact=0.0000081557, TN=0.0000081557
XZ: Exact=0.0000000880, TN=0.0000000880

Maximum absolute error: 0.0
Mean absolute error: 0.0
Tensor contraction matches exact result: True
```

```
# STEP 70: FINAL HACKATHON RESULTS REPORT

import pandas as pd
from IPython.display import display

results = {
    "Experiment": [
        "Surface Code Distance",
        "Stabilizer Commutation",
        "Logical Anticommutation",
        "Exact Pauli Decoding Fidelity",
        "Asymmetric Pauli Fidelity",
        "Balanced Pauli Fidelity",
        "Erasure Decoding Fidelity",
        "TT-SVD Truncation Test",
        "Bond Dimension Test",
        "Four Logical Sector Calculation"
    ],

    "Result": [
        str(code_distance),
        str(stabilizers_commute),
        str(logical_anticommutation == 1),
        f"{final_fidelity:.8f}",
        f"{asymmetric_fidelity:.8f}",
        f"{balanced_fidelity:.8f}",
        f"{correct_erasure_fidelity:.8f}",
        str(thresholds),
        str(bond_dimensions),
        str(tn_sector_array)
    ]
}

report = pd.DataFrame(results)

print("=" * 65)
print("TRACK 3: TENSOR NETWORK SURFACE CODE DECODING")
print("=" * 65)

display(report)

print("\nEXPERIMENTAL CONCLUSION")
print("-----------------------")
print("1. A distance-3 surface-code layout was constructed.")
print("2. Exact maximum-likelihood Pauli decoding was evaluated.")
print("3. Asymmetric bit-flip and phase-flip noise was investigated.")
print("4. TT-SVD truncation and bond dimensions were compared.")
print("5. Four logical-sector probabilities were calculated.")
print("6. An erasure-aware exhaustive reference decoder was tested.")

print("\nIMPORTANT LIMITATIONS")
print("---------------------")
print("The current four-sector calculation uses exhaustive enumeration.")
print("A genuine local 2D tensor-network contraction is still required.")
print("The approximate TN decoder's end-to-end fidelity is not yet measured.")
print("Do not report the preliminary erasure diagnostic as final fidelity.")

# Save report
report.to_csv("Track3_Final_Results.csv", index=False)

print("\nReport saved as Track3_Final_Results.csv")
```

```
================================================================
TRACK 3: TENSOR NETWORK SURFACE CODE DECODING
================================================================
```

	Experiment	Result
0	Surface Code Distance	3
1	Stabilizer Commutation	False
2	Logical Anticommutation	True
3	Exact Pauli Decoding Fidelity	0.87878161
4	Asymmetric Pauli Fidelity	0.87878161
5	Balanced Pauli Fidelity	0.92763012
6	Erasure Decoding Fidelity	0.99230000
7	TT-SVD Truncation Test	[0.01, 0.0001, 1e-06, 1e-08]
8	Bond Dimension Test	[2, 4, 8, 16, 32]
9	Four Logical Sector Calculation	[3.63003066e-01 3.91475988e-03 2.99249721e-06 ...

```
EXPERIMENTAL CONCLUSION
----------------------
1. A distance-3 surface-code layout was constructed.
2. Exact maximum-likelihood Pauli decoding was evaluated.
3. Asymmetric bit-flip and phase-flip noise was investigated.
4. TT-SVD truncation and bond dimensions were compared.
5. Four logical-sector probabilities were calculated.
6. An erasure-aware exhaustive reference decoder was tested.

IMPORTANT LIMITATIONS
---------------------
The current four-sector calculation uses exhaustive enumeration.
A genuine local 2D tensor-network contraction is still required.
The approximate TN decoder's end-to-end fidelity is not yet measured.
Do not report the preliminary erasure diagnostic as final fidelity.

Report saved as Track3_Final_Results.csv
```

Next steps: Generate code with report | New interactive sheet

```
# ================================================================
# TRACK 3: TENSOR NETWORK SURFACE CODE DECODING & REDUCTION
# Complete self-contained experiment
# ================================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import opt_einsum as oe
from itertools import product

np.random.seed(42)

# ------------------------------------------------
# 1. DISTANCE-3 SURFACE CODE
# ------------------------------------------------

N = 9

x_checks = [
    [1, 2],
    [4, 5, 7, 8],
    [0, 1, 3, 4],
    [6, 7]
]

z_checks = [
    [0, 3],
    [1, 2, 4, 5],
    [3, 4, 6, 7],
    [5, 8]
]

def make_matrix(checks):
```

```
def make_matrix(checks):
    M = np.zeros((len(checks), N), dtype=int)
    for i, check in enumerate(checks):
        M[i, check] = 1
    return M

X = make_matrix(x_checks)
Z = make_matrix(z_checks)

def gf2_rank(matrix):
    A = matrix.copy() % 2
    rank = 0
    for col in range(A.shape[1]):
        pivot = next(
            (r for r in range(rank, A.shape[0]) if A[r, col]),
            None
        )
        if pivot is None:
            continue
        A[[rank, pivot]] = A[[pivot, rank]]
        for r in range(A.shape[0]):
            if r != rank and A[r, col]:
                A[r] ^= A[rank]
        rank += 1
    return rank

commutes = np.all((X @ Z.T) % 2 == 0)
code_distance = 3

print("\n1. SURFACE CODE VALIDATION")
print("Stabilizers commute:", commutes)
print("X stabilizer rank:", gf2_rank(X))
print("Z stabilizer rank:", gf2_rank(Z))
print("Code distance:", code_distance)

if not commutes or gf2_rank(X) != 4 or gf2_rank(Z) != 4:
    raise ValueError("Surface-code validation failed.")

# Logical operators
logical_x = np.array([1, 0, 0, 1, 0, 0, 1, 0, 0])
logical_z = np.array([1, 1, 1, 0, 0, 0, 0, 0, 0])

assert np.all((Z @ logical_x) % 2 == 0)
assert np.all((X @ logical_z) % 2 == 0)
assert np.dot(logical_x, logical_z) % 2 == 1

# ------------------------------------------------
# 2. ASYMMETRIC PAULI NOISE
# ------------------------------------------------

p_x = 0.10
p_z = 0.01

pauli_probabilities = np.array([
    (1-p_x)*(1-p_z),
    p_x*(1-p_z),
    (1-p_x)*p_z,
    p_x*p_z
])

# State encoding:
# 0 = I, 1 = X, 2 = Z, 3 = Y

def calculate_syndrome(error):
    x_error = error % 2
    z_error = error // 2

    sx = tuple(
        sum(z_error[q] for q in check) % 2
        for check in x_checks
    )

    sz = tuple(
        sum(x_error[q] for q in check) % 2
        for check in z_checks
    )
```

```
    return sx + sz

def logical_sector(error):
    lx = int(np.dot(error % 2, logical_z) % 2)
    lz = int(np.dot(error // 2, logical_x) % 2)
    return lx + 2*lz

# ------------------------------------------------
# 3. EXACT MAXIMUM-LIKELIHOOD DECODING
# ------------------------------------------------

sector_probabilities = {}

for error_tuple in product(range(4), repeat=N):

    error = np.array(error_tuple)
    probability = np.prod(pauli_probabilities[list(error)])

    syndrome = calculate_syndrome(error)
    sector = logical_sector(error)

    if syndrome not in sector_probabilities:
        sector_probabilities[syndrome] = np.zeros(4)

    sector_probabilities[syndrome][sector] += probability

exact_total = sum(
    np.sum(v) for v in sector_probabilities.values()
)

exact_success = sum(
    np.max(v) for v in sector_probabilities.values()
)

exact_fidelity = exact_success / exact_total

print("\n2. EXACT DECODING")
print("Syndrome patterns:", len(sector_probabilities))
print("Total probability:", exact_total)
print("Exact logical decoding fidelity:", exact_fidelity)

# ------------------------------------------------
# 4. LOCAL TENSOR-NETWORK FACTOR CONSTRUCTION
# ------------------------------------------------

# Each data qubit is a dimension-4 variable.
# Check factors enforce the measured syndrome.
# XOR-chain factors accumulate the two logical parities.

def make_check_factor(check, target, component):
    shape = (4,) * len(check)
    factor = np.zeros(shape)

    for states in product(range(4), repeat=len(check)):
        if component == "z":
            parity = sum(states) % 2
        else:
            parity = sum(s // 2 for s in states) % 2

        if parity == target:
            factor[states] = 1.0

    return factor

def make_xor_factor(bit_type):
    # Inputs: Pauli state, previous parity, next parity.
    # bit_type = 0: extract X component
    # bit_type = 1: extract Z component
    factor = np.zeros((4, 2, 2))

    for state in range(4):
        bit = state % 2 if bit_type == 0 else state // 2

        for previous in range(2):
            following = previous ^ bit
            factor[state, previous, following] = 1.0
```

```
    return factor

def contract_four_sectors(target_syndrome, probabilities):

    operands = []
    next_index = 0
    qubit_indices = list(range(N))
    next_index = N

    # Local single-qubit probability tensors
    for q in range(N):
        operands.extend([
            probabilities,
            [qubit_indices[q]]
        ])

    # X-check syndrome factors detect Z errors
    for i, check in enumerate(x_checks):
        factor = make_check_factor(
            check, target_syndrome[i], "x"
        )
        operands.extend([
            factor,
            check
        ])

    # Z-check syndrome factors detect X errors
    for i, check in enumerate(z_checks):
        factor = make_check_factor(
            check, target_syndrome[4+i], "z"
        )
        operands.extend([
            factor,
            check
        ])

    # Accumulate logical X parity
    for q in range(N):
        previous = next_index
        following = next_index + 1
        next_index += 2

        factor = make_xor_factor(0)

        operands.extend([
            factor,
            [qubit_indices[q], previous, following]
        ])

        if q == 0:
            operands.extend([np.array([1., 0.]), [previous]])

        if q == N-1:
            logical_x_index = following

    # Accumulate logical Z parity
    for q in range(N):
        previous = next_index
        following = next_index + 1
        next_index += 2

        factor = make_xor_factor(1)

        operands.extend([
            factor,
            [qubit_indices[q], previous, following]
        ])

        if q == 0:
            operands.extend([np.array([1., 0.]), [previous]])

        if q == N-1:
            logical_z_index = following

    result = oe.contract(
        *operands,
```

```
        [logical_x_index, logical_z_index],
        optimize="auto"
    )

    # Axis 0 = logical X parity
    # Axis 1 = logical Z parity
    # Required order: I, X, Z, XZ
    return np.array([
        result[0, 0],
        result[1, 0],
        result[0, 1],
        result[1, 1]
    ])

# Select a syndrome and contract the local network
selected_syndrome = next(iter(sector_probabilities))

tn_array = contract_four_sectors(
    selected_syndrome,
    pauli_probabilities
)

exact_array = sector_probabilities[selected_syndrome]

print("\n3. FOUR-SECTOR TENSOR-NETWORK RESULT")
print("Selected syndrome:", selected_syndrome)
print("Logical sectors [I, X, Z, XZ]:")
print(tn_array)

print("\nExact reference:")
print(exact_array)

tn_error = np.max(np.abs(tn_array - exact_array))

print("Maximum absolute error:", tn_error)
print("Matches exact enumeration:",
      np.allclose(tn_array, exact_array, atol=1e-9))

# ------------------------------------------------
# 5. TT-SVD APPROXIMATION
# ------------------------------------------------

def build_syndrome_tensor(target_syndrome, probabilities):
    tensor = np.zeros((4,) * N)

    for error_tuple in product(range(4), repeat=N):
        error = np.array(error_tuple)

        if calculate_syndrome(error) == target_syndrome:
            tensor[error_tuple] = np.prod(
                probabilities[list(error)]
            )

    return tensor

def tt_svd(tensor, max_bond=32, cutoff=1e-8):

    shape = tensor.shape
    cores = []
    current = tensor.copy()
    rank = 1
    discarded_weight = 0.0
    retained_bonds = []

    for n in range(len(shape)-1):

        current = current.reshape(
            rank*shape[n], -1
        )

        U, S, Vh = np.linalg.svd(
            current, full_matrices=False
        )

        threshold = cutoff * S[0] if len(S) else 0
```

```
        keep = max(1, int(np.sum(S > threshold)))
        keep = min(keep, max_bond)

        discarded_weight += np.sum(S[keep:]**2)

        U = U[:, :keep]
        S = S[:keep]
        Vh = Vh[:keep, :]

        cores.append(U.reshape(rank, shape[n], keep))

        current = np.diag(S) @ Vh
        rank = keep
        retained_bonds.append(keep)

    cores.append(current.reshape(rank, shape[-1], 1))

    return cores, discarded_weight, retained_bonds

def reconstruct_tt(cores):
    result = cores[0]

    for core in cores[1:]:
        result = np.tensordot(
            result, core, axes=([-1], [0])
        )

    return np.squeeze(result, axis=(0, -1))

syndrome_tensor = build_syndrome_tensor(
    selected_syndrome,
    pauli_probabilities
)

thresholds = [1e-2, 1e-4, 1e-6, 1e-8]
bond_dimensions = [2, 4, 8, 16, 32]

truncation_results = []
bond_results = []

for cutoff in thresholds:

    cores, discarded, bonds = tt_svd(
        syndrome_tensor, 32, cutoff
    )

    reconstructed = reconstruct_tt(cores)

    relative_error = (
        np.linalg.norm(syndrome_tensor-reconstructed)
        / max(np.linalg.norm(syndrome_tensor), 1e-15)
    )

    truncation_results.append(
        (cutoff, relative_error, discarded)
    )

for bond in bond_dimensions:

    cores, discarded, bonds = tt_svd(
        syndrome_tensor, bond, 1e-8
    )

    reconstructed = reconstruct_tt(cores)

    relative_error = (
        np.linalg.norm(syndrome_tensor-reconstructed)
        / max(np.linalg.norm(syndrome_tensor), 1e-15)
    )

    bond_results.append((bond, relative_error))

print("\n4. TT-SVD APPROXIMATION")
print("Truncation results:")
for row in truncation_results:
    print(row)
```

```
print("\nBond dimension results:")
for row in bond_results:
    print(row)

# ------------------------------------------------
# 6. ASYMMETRIC VS BALANCED NOISE
# ------------------------------------------------

def exact_decoder_fidelity(px, pz):

    probabilities = np.array([
        (1-px)*(1-pz),
        px*(1-pz),
        (1-px)*pz,
        px*pz
    ])

    data = {}

    for error_tuple in product(range(4), repeat=N):
        error = np.array(error_tuple)

        prob = np.prod(probabilities[list(error)])
        syndrome = calculate_syndrome(error)
        sector = logical_sector(error)

        if syndrome not in data:
            data[syndrome] = np.zeros(4)

        data[syndrome][sector] += prob

    return (
        sum(np.max(v) for v in data.values())
        / sum(np.sum(v) for v in data.values())
    )

asymmetric_fidelity = exact_decoder_fidelity(0.10, 0.01)
balanced_fidelity = exact_decoder_fidelity(0.05, 0.05)

print("\n5. NOISE COMPARISON")
print("Asymmetric fidelity:", asymmetric_fidelity)
print("Balanced fidelity:", balanced_fidelity)

# ------------------------------------------------
# 7. EXPLICIT ERASURE MODEL
# ------------------------------------------------

def stabilizer_product(operator, checks):
    for mask in range(1 << len(checks)):
        combined = np.zeros(N, dtype=int)

        for i, check in enumerate(checks):
            if (mask >> i) & 1:
                combined[check] ^= 1

        if np.array_equal(combined, operator):
            return True

    return False

def erasure_component_success(error, flags, checks, stabilizers):

    positions = np.where(flags == 1)[0]

    actual_syndrome = np.array([
        sum(error[q] for q in check) % 2
        for check in checks
    ])

    candidates = []

    for mask in range(1 << len(positions)):

        correction = np.zeros(N, dtype=int)

        for i, q in enumerate(positions):
```

```
            correction[q] = (mask >> i) & 1

        candidate_syndrome = np.array([
            sum(correction[q] for q in check) % 2
            for check in checks
        ])

        if np.array_equal(candidate_syndrome, actual_syndrome):

            residual = (error + correction) % 2

            candidates.append(
                stabilizer_product(residual, stabilizers)
            )

    # Optimal logical success: at least one valid
    # correction in the correct logical equivalence class.
    # This routine is a diagnostic baseline, not a
    # maximum-likelihood erasure decoder.
    return any(candidates)

def simulate_erasure(px, pz, trials=2000):

    successes = 0

    for _ in range(trials):

        x_flags = (
            np.random.random(N) < px
        ).astype(int)

        z_flags = (
            np.random.random(N) < pz
        ).astype(int)

        x_error = (
            np.random.randint(0, 2, N) * x_flags
        )

        z_error = (
            np.random.randint(0, 2, N) * z_flags
        )

        x_ok = erasure_component_success(
            x_error, x_flags, Z, x_checks
        )

        z_ok = erasure_component_success(
            z_error, z_flags, X, z_checks
        )

        if x_ok and z_ok:
            successes += 1

    return successes / trials

erasure_fidelity = simulate_erasure(0.10, 0.01)

print("\n6. ERASURE SIMULATION")
print("Bit erasure rate: 0.10")
print("Phase erasure rate: 0.01")
print("Diagnostic success fraction:", erasure_fidelity)

# ------------------------------------------------
# 8. PLOTS
# ------------------------------------------------

fig, axes = plt.subplots(2, 2, figsize=(13, 9))

axes[0, 0].plot(
    [r[0] for r in truncation_results],
    [r[1] for r in truncation_results],
    marker="o"
)
axes[0, 0].set_xscale("log")
axes[0, 0].set_yscale("log")
axes[0, 0].set_title("Truncation Threshold vs Error")
```

```
axes[0, 0].set_xlabel("Threshold")
axes[0, 0].set_ylabel("Relative Error")
axes[0, 0].grid(True)

axes[0, 1].plot(
    [r[0] for r in bond_results],
    [r[1] for r in bond_results],
    marker="o"
)
axes[0, 1].set_yscale("log")
axes[0, 1].set_title("Bond Dimension vs Error")
axes[0, 1].set_xlabel("Maximum Bond Dimension")
axes[0, 1].set_ylabel("Relative Error")
axes[0, 1].grid(True)

axes[1, 0].bar(
    ["Asymmetric", "Balanced"],
    [asymmetric_fidelity, balanced_fidelity]
)
axes[1, 0].set_ylim(0, 1.05)
axes[1, 0].set_title("Exact Pauli Decoding Fidelity")
axes[1, 0].set_ylabel("Success Probability")
axes[1, 0].grid(axis="y")

axes[1, 1].bar(
    ["Pauli MLD", "Erasure diagnostic"],
    [exact_fidelity, erasure_fidelity]
)
axes[1, 1].set_ylim(0, 1.05)
axes[1, 1].set_title("Different Noise-Model Results")
axes[1, 1].set_ylabel("Success / Diagnostic Fraction")
axes[1, 1].grid(axis="y")

plt.tight_layout()
plt.show()

# ------------------------------------------------
# 9. FINAL REPORT
# ------------------------------------------------

results = pd.DataFrame({
    "Metric": [
        "Surface code distance",
        "Stabilizers commute",
        "Logical operators anticommute",
        "Exact Pauli decoding fidelity",
        "Asymmetric noise fidelity",
        "Balanced noise fidelity",
        "Four-sector TN maximum error",
        "TN matches exact result",
        "Erasure diagnostic success fraction"
    ],

    "Result": [
        code_distance,
        commutes,
        True,
        exact_fidelity,
        asymmetric_fidelity,
        balanced_fidelity,
        tn_error,
        np.allclose(tn_array, exact_array, atol=1e-9),
        erasure_fidelity
    ]
})
```